

Dossier de Projet

SkillSwap

Plateforme communautaire d'échange de compétences

Présenté par

Jérémy Soriano

en vue de l'obtention du

Titre Professionnel Concepteur Développeur d'Applications

(Niveau 6 — RNCP)

École O'clock — Promotion Dublin

Soutenance : Mercredi 26 août 2026

Application en production : <https://skill-swap.fr>

Documentation technique (extension post-projet) : <https://skillswap-docs.vercel.app>

Sommaire

Sommaire	1
1. Introduction	4
1.1. Présentation du candidat	4
2. Compétences du référentiel CDA couvertes	5
2.1. AT1 — Développer une application sécurisée	5
2.1.1. CP1 — Installer et configurer son environnement de travail	5
2.1.2. CP2 — Développer des interfaces utilisateur	5
2.1.3. CP3 — Développer des composants métier	5
2.1.4. CP4 — Contribuer à la gestion d'un projet informatique	5
2.2. AT2 — Concevoir et développer une application sécurisée organisée en couches	5
2.2.1. CP5 — Analyser les besoins et maquetter une application	5
2.2.2. CP6 — Définir l'architecture logicielle d'une application	6
2.2.3. CP7 — Concevoir et mettre en place une base de données relationnelle	6
2.2.4. CP8 — Développer des composants d'accès aux données SQL et NoSQL	6
2.3. AT3 — Préparer le déploiement d'une application sécurisée	7
2.3.1. CP9 — Préparer et exécuter les plans de tests d'une application ...	7
2.3.2. CP10 — Préparer et documenter le déploiement d'une application ...	7
2.3.3. CP11 — Contribuer à la mise en production dans une démarche DevOps	7
3. Cahier des charges	8
3.1. Présentation du projet	8
3.2. Public cible	8
3.3. Besoins fonctionnels — le MVP	8
3.4. Évolutions envisagées et périmètre effectivement livré	8
3.5. Périmètre fonctionnel détaillé — user stories	9
3.6. Règles de gestion arbitrées en cadrage	9
3.7. Contraintes techniques et de compatibilité	10
4. Présentation du commanditaire et de l'équipe	11
4.1. O'clock — école de développement à distance synchrone	11
4.2. L'apothéose — dispositif simulant l'entreprise	11
4.3. L'équipe SkillSwap	11
4.4. Le projet	11
5. Gestion de projet	12
5.1. Cadre et organisation de l'équipe	12
5.2. Méthodologie : Scrum adapté à quatre semaines	12
5.3. Le sprint 0 : cadrage et conception	12
5.4. Déroulé des sprints	12
5.5. Ma contribution et sa progression	13
5.6. Outils et flux de travail	13
5.7. Anticipation des risques	13

6.	Spécifications fonctionnelles	14
6.1.	Contraintes du projet et livrables attendus	14
6.1.1.	Contraintes	14
6.1.2.	Livrables attendus	15
6.2.	Architecture logicielle du projet	16
6.2.1.	Communications inter-composants	17
6.3.	Maquettes et enchaînement des maquettes	18
6.3.1.	Arborescence de l'application	19
6.3.2.	Parcours utilisateur principal	22
6.3.3.	Captures de maquettes Figma	23
6.4.	Modèle entités-associations et modèle physique de la base de données . .	23
6.4.1.	Domaines fonctionnels du modèle	25
6.5.	Script de création et de modification de la base de données	26
6.6.	Diagramme du comportement des fonctionnalités — cas d'utilisation	29
6.6.1.	Acteurs et cas d'utilisation principaux	29
6.7.	Diagramme du détail du cas d'utilisation le plus significatif	30
6.7.1.	Lecture du diagramme	31
7.	Spécifications techniques	33
7.1.	Stack technique	33
7.2.	Choix architecturaux — synthèse des ADRs	33
7.3.	Patterns transversaux	35
7.4.	Sécurité transversale	35
8.	Réalisations — Messagerie temps réel	37
8.1.	Captures d'écran d'interfaces utilisateur	38
8.1.1.	Liste des conversations — vue desktop	38
8.1.2.	Thread de message ouvert — vue desktop	39
8.1.3.	Dialogue d'évaluation après clôture	39
8.2.	Composants métier — orchestrateur <code>useMessaging</code> et optimistic UI	40
8.2.1.	Architecture des hooks — le pattern de composition	40
8.2.2.	Hook racine — composition	41
8.2.3.	Optimistic UI — gestion du <code>tempId</code> négatif	43
8.3.	Composants d'accès aux données — services Prisma	44
8.3.1.	Création atomique d'un message + mise à jour de la conversation .	44
8.3.2.	Pagination cursor-based des messages	45
8.3.3.	Schéma Prisma — extrait des modèles principaux	45
8.4.	Autres composants — Socket.IO server et middleware métier	46
8.4.1.	Authentification Socket.IO par cookie JWT	46
8.4.2.	Middleware métier <code>requireSimpleFollow</code> — gating de la création de conversation	47
8.4.3.	Modèle de rooms Socket.IO	48
8.4.4.	Diagramme de séquence — envoi d'un message	48
8.5.	Bilan technique de la section	49
8.5.1.	Dette technique assumée	50
9.	Éléments de sécurité	51
9.1.	Authentification du canal Socket.IO	51
9.2.	Contrôles d'accès appliqués à chaque event	51

9.3.	Défense en profondeur — front, back, base	52
9.4.	Dettes de sécurité assumées	53
10.	Plan de tests	54
10.1.	Stratégie de tests	54
10.2.	Tests backend (Node Test Runner natif)	54
10.3.	Absence de tests automatisés côté frontend	54
10.4.	Plan de tests fonctionnels	54
10.5.	Manques honnêtes (à mentionner)	55
11.	Jeu d'essai	56
11.1.	Scénario testé	56
11.2.	Étape 1 — Alice suit Bob (POST /follows/:bobId/follow)	56
11.3.	Étape 2 — Alice crée la conversation (POST /api/v1/conversations)	56
11.4.	Étape 3 — Alice envoie le premier message via Socket.IO	56
11.5.	Étape 4 — Bob reçoit la notification temps réel	57
11.6.	Étape 5 — Optimistic UI résolu côté Alice	57
11.7.	Captures d'écran annexées	58
11.8.	Limites du jeu d'essai	58
12.	Veille de sécurité	59
12.1.	Périmètre et constat	59
12.2.	Une vulnérabilité identifiée et traitée pendant le projet	59
12.3.	Mesures de sécurité retenues pendant le projet	59
12.4.	Analyse OWASP conduite sur l'application	60
12.5.	Ce que je mettrais en place	60
13.	Difficultés rencontrées et solutions	61
13.1.	La mise au point de l'environnement conteneurisé	61
13.2.	Le contrat d'interface entre le frontend et le backend	61
13.3.	Une revue de code à mi-parcours	61
13.4.	Le cycle de vie de la session	62
13.5.	Les tests d'intégration backend	62
13.6.	Ce que ces difficultés ont en commun	62
14.	Conclusion	63
14.1.	Bilan du projet	63
14.2.	Évolutions identifiées pour la V2	63
14.3.	Apprentissage personnel	63
15.	Lexique	65
16.	Annexes	66
16.1.	Annexe A — Schéma Prisma de la fonctionnalité messagerie	66
16.1.1.	Extraits SQL générés par Prisma — migrations clés	67
16.2.	Annexe B — Handler Socket.IO <code>message:send</code> (code complet)	68
16.3.	Annexe C — Orchestrateur frontend <code>useMessaging</code> (139 LOC, complet)	71
16.4.	Annexe D — Composants UI clés de la messagerie	74
16.4.1.	<code>MessageList.tsx</code> — affichage virtualisé du fil de discussion	74
16.4.2.	<code>MessageInput.tsx</code> — saisie utilisateur et envoi	76
16.5.	Annexe E — Plan de tests détaillé de la messagerie	77
16.6.	Annexe F — Maquettes Figma de la messagerie	77
16.7.	Liens utiles	78

1. Introduction

Ce dossier de projet présente SkillSwap, plateforme web d'échange de compétences entre particuliers, développée dans le cadre de l'apothéose finale du Titre Professionnel Concepteur Développeur d'Applications. Le projet a été porté pendant quatre semaines par une équipe de cinq apprenants de la promotion Dublin.

J'y ai tenu le rôle de Lead Front : architecture et développement des interfaces, conception des maquettes, chaîne de déploiement de production, et documentation technique du projet.

Le dossier suit la structure du plan-type : recensement des onze compétences professionnelles mobilisées (section 2), cadrage du besoin (section 3), présentation du commanditaire pédagogique et de l'équipe (section 4), méthodologie de gestion de projet (section 5), spécifications fonctionnelles et techniques (section 6 et section 7), description détaillée de la fonctionnalité représentative — la messagerie temps réel — sous l'angle des réalisations (section 8) et de la sécurité (section 9), plan de tests (section 10), jeu d'essai (section 11), démarche de veille (section 12), retour d'expérience sur les difficultés rencontrées (section 13), conclusion ouvrant sur les évolutions V2 (section 14), lexique (section 15), et annexes regroupant les extraits de code et artefacts visuels.

Un parti pris a guidé sa rédaction : chaque affirmation factuelle y est adossée à une source vérifiable dans le code livré. Les artefacts de modélisation ont été reconstruits de façon déterministe depuis la base réelle, les éléments produits après la période de projet sont explicitement datés, et les dettes techniques identifiées sont documentées plutôt que masquées.

Le projet est aujourd'hui en production sur skill-swap.fr.

1.1. Présentation du candidat

Mon parcours vers le développement est celui d'une reconversion. Après un BTS Management Commercial Opérationnel et un Bachelor Responsable Manager, j'ai exercé six ans en commerce et en management : conseiller client, puis responsable du développement de la livraison à domicile pour une enseigne de grande distribution, enfin commercial dans l'assurance. Ces années m'ont donné une lecture métier des problèmes techniques — l'habitude de chercher le besoin réel avant la solution, et le sens des priorités d'une activité.

C'est cette même exigence de compréhension du besoin qui m'a conduit vers l'ingénierie logicielle. J'ai intégré la formation Concepteur Développeur d'Applications d'O'clock pour acquérir des fondations solides : architecture, conception de bases de données, développement full-stack et mise en production. L'apothéose SkillSwap constitue l'aboutissement de ce cursus — un projet d'équipe mené de bout en bout, du cadrage au déploiement.

2. Compétences du référentiel CDA couvertes

2.1. AT1 — Développer une application sécurisée

2.1.1. CP1 — Installer et configurer son environnement de travail

L'équipe s'est dotée d'un outillage qualité (ESLint, Prettier, Husky avec hook de pre-commit), mis en place par un coéquipier. J'ai contribué à la configuration Docker de développement (`backend/Dockerfile.dev`, `docker-compose.dev.yml`) et porté les correctifs de setup (PR `fix-setup-docker`). → §5.1.

2.1.2. CP2 — Développer des interfaces utilisateur

J'ai développé les interfaces de la page d'accueil (89 %), de la recherche (`SearchPage`, 80 %), de la consultation de profil public (`ProfileFull`, `ProfileTeaser`, 100 %) et de la navigation (`DesktopNav` 100 %, `MobileNav` 92 %), dans une architecture Atomic Design. Les dialogues d'édition de profil et le formulaire d'authentification ont été portés par des coéquipiers. → §6.3, annexes.

2.1.3. CP3 — Développer des composants métier

J'ai développé plusieurs hooks métier React : la gestion d'état de formulaire (`useFormState`, 100 %), la consommation de l'API de recherche avec debounce, pagination et annulation par `AbortController` (`useSearch`, 88 %), et la récupération des membres suivis (`useFollowedUsers`, 86 %). L'orchestration de la messagerie temps réel (`useMessaging` et les hooks socket) a été portée par un coéquipier. → §6, annexes.

2.1.4. CP4 — Contribuer à la gestion d'un projet informatique

J'ai travaillé en flux de Pull Requests sur branches de feature — six branches portées (SEO, page profil, recherche, setup Docker), treize Pull Requests intégrées, deuxième contributeur en volume de commits — au sein d'une organisation Scrum d'équipe (quatre sprints, rituels agiles, backlog Trello). → §5.

2.2. AT2 — Concevoir et développer une application sécurisée organisée en couches

2.2.1. CP5 — Analyser les besoins et maquetter une application

J'ai formalisé les cas d'utilisation et le parcours utilisateur de la plateforme en sprint 0 (`docs/uml/user/use-cases.puml`, `user-flow.puml`), et j'ai conduit une démarche d'analyse de besoin écrite pour la fonctionnalité de profil public : contexte, problématique, stratégie et spécification fonctionnelle formalisés avant l'implémentation (`frontend/src/.code-review/profil-teaser-strategy.md`, 601 lignes). Cette analyse débouche directement sur le composant d'accès aux données décrit en CP8, ce qui matérialise une chaîne complète besoin → spécification → implémentation. → détail §6.3, §6.6.

2.2.2. CP6 — Définir l'architecture logicielle d'une application

J'ai conçu la chaîne de déploiement de production (reverse proxy Nginx, `docker-compose.prod`, `Dockerfile.prod` multi-étapes) — `devops/` à 76 %, dont `nginx/prod.conf` et `Dockerfile.prod` quasi intégralement. Côté applicatif, j'ai contribué à l'implémentation de l'architecture frontend en Atomic Design. Concernant l'architecture backend en couches (router → controller → service → ORM), je ne l'ai pas implémentée, mais je l'ai analysée et outillée : j'ai vérifié mécaniquement le respect du sens de dépendance (`dependency-cruiser`, 4 règles, 0 violation) et documenté les cinq accès Prisma hors couche service — dont le principal, `realtime/socket.ts`, qui persiste les messages directement via `prisma.message.create` (`socket.ts:245`) au lieu de passer par `message.service.ts`, dupliquant la logique du chemin REST ; j'ai aussi relevé que cette persistance est un `Promise.all` (`socket.ts:244`) et non une transaction. → détail §6.2, §7.

2.2.3. CP7 — Concevoir et mettre en place une base de données relationnelle

J'ai produit le modèle entité-relation initial de la plateforme en sprint 0 (`docs/uml/erd.puml`), antérieur à l'implémentation du schéma, et j'ai contribué directement au schéma en ajoutant le champ `slug` à l'entité `Category` (migration `add_category_slug`). Au-delà de cette contribution d'écriture, je maîtrise l'intégralité de la modélisation, que j'ai reconstruite et vérifiée de façon déterministe depuis la base de production : MCD Merise (notation Mocodo), MLD (14 tables, DBML), MPD (SQL des six migrations) et ERD. Cette modélisation couvre la normalisation jusqu'en 3NF — avec une dénormalisation assumée de `message.receiver_id` pour la diffusion temps réel —, les clés composites des tables de jonction, les contraintes d'unicité par paire (`follow`, `evaluation`), et la distinction entre contraintes d'intégrité (FK NOT NULL) et contraintes applicatives (la règle des deux participants d'une conversation, désormais (0,2)). → détail §6.4 et §6.5.

2.2.4. CP8 — Développer des composants d'accès aux données SQL et NoSQL

J'ai développé le composant d'accès aux données du profil public (`getPublicProfileService`, `profile.service.ts:20-88`, ainsi que la tranche verticale route → service → rendu SSR → composant, 100 %). Ce composant met en œuvre une projection maîtrisée : un `select` racine qui énumère explicitement les colonnes remontées (`:31, :33`), et un chargement ciblé des relations — compétences avec leur catégorie via un `include` imbriqué (`:40-44`), évaluations restreintes au seul champ `score` (`:45-49`). S'y ajoutent une agrégation applicative (moyenne des avis arrondie au dixième, `:61-65`), et une minimisation des données exposées (description tronquée `:69-70`, nom réduit à l'initiale `:78`). J'ai également développé le composant frontend de consommation de l'API de recherche (`hooks/useSearch.ts`, 88 % : debounce, pagination, annulation par `AbortController`). L'accès aux données NoSQL (Meilisearch) a été porté par le Lead Back. → détail §8.3.

2.3. AT3 — Préparer le déploiement d'une application sécurisée

2.3.1. CP9 — Préparer et exécuter les plans de tests d'une application

L'équipe a mis en place sept suites de tests d'intégration backend exécutées par le Node Test Runner natif, écrites par trois coéquipiers. Le frontend, dont j'étais le principal contributeur, n'a pas été couvert par des tests automatisés — c'est la première dette que j'identifie, et la stratégie de test frontend (Vitest, Playwright) reste à l'état de décision documentée. → §10.

2.3.2. CP10 — Préparer et documenter le déploiement d'une application

J'ai conçu la chaîne de déploiement de production : images Docker multi-étapes (`Dockerfile.prod` front 100 %, back 86 %), orchestration `docker-compose.prod` (82 %), variables d'environnement (`.env.prod.example`, 100 %), le tout documenté dans `devops/README.md` (92 %, 323 lignes). → §7, dépôt `devops/`.

2.3.3. CP11 — Contribuer à la mise en production dans une démarche DevOps

J'ai conçu la chaîne de mise en production : reverse proxy Nginx avec TLS, HSTS et en-têtes de sécurité (`nginx/prod.conf`, 99 %), orchestration Docker Compose et images multi-étapes. L'automatisation du déploiement continu (pipeline CD) et l'exécution de la mise en production réelle ont été portées par deux coéquipiers. → §7.

3. Cahier des charges

Le cadrage du projet a été formalisé dans un brief d'équipe, complété d'un tableur de user stories et d'un journal des questions de conception¹. La présente section en restitue le contenu.

3.1. Présentation du projet

SkillSwap est une plateforme web d'échange de compétences entre particuliers. Chaque utilisateur peut proposer ses savoir-faire et bénéficier de ceux des autres, favorisant l'entraide et le partage de connaissances. La plateforme permet de créer un profil proposant ses services grâce à ses compétences, de rechercher d'autres profils ayant une compétence souhaitée, et une mise en relation via une messagerie intégrée. Elle repose sur une architecture moderne avec frontend et backend séparés, conteneurisée avec Docker pour garantir portabilité, sécurité et scalabilité.

3.2. Public cible

Des particuliers âgés de 18 à 60 ans, à l'aise avec les outils numériques, intéressés par l'entraide et l'économie collaborative. Ils cherchent à acquérir de nouvelles compétences ou à résoudre des besoins ponctuels (bricolage, informatique, jardinage, cuisine) sans passer par des services professionnels payants ; en échange, ils sont prêts à partager leurs propres compétences. Exemples : un développeur ayant besoin d'une réparation automobile, un jardinier souhaitant rénover sa salle de bain, un bricoleur désireux d'apprendre à cuisiner.

3.3. Besoins fonctionnels — le MVP

- Landing page avec la présentation de SkillSwap et quelques profils.
- Système d'inscription et de connexion.
- Gestion du profil utilisateur détaillé avec ses compétences, intérêts et disponibilités.
- Moteur de recherche par compétences afin de trouver de potentiels profils correspondants.
- Possibilité de suivre / ne plus suivre un profil.
- Possibilité de rentrer en contact avec un profil pour entamer un échange sur les compétences mutuelles.
- Possibilité d'évaluer un partenaire après échange de compétences.

3.4. Évolutions envisagées et périmètre effectivement livré

Le cadrage identifiait plusieurs évolutions possibles au-delà du MVP : la messagerie instantanée via WebSockets, la recherche avancée avec Meilisearch, le blocage d'un profil, la suggestion automatique de profils compatibles et la création de groupes ou

¹Documents produits hors dépôt Git, comme le journal de bord mentionné en section 5. Leur production est attestée par la checklist de conception validée par l'équipe pédagogique, qui coche notamment « user stories » et « dictionnaire de données ».

communautés. Deux d'entre elles ont finalement été livrées dans le périmètre du projet : la messagerie temps réel (Socket.IO) et le moteur de recherche Meilisearch. Les autres — blocage de profil, suggestions automatiques, groupes — n'ont pas été implémentées et restent des perspectives d'évolution.

3.5. Périmètre fonctionnel détaillé — user stories

Acteur	« Je souhaite pouvoir ... afin de ... »
Visiteur	Accéder à la landing page, afin de prévisualiser le fonctionnement du site.
Visiteur	Accéder au formulaire de création de compte.
Visiteur	Accéder au formulaire de connexion.
Membre	Accéder à la page de recherche des profils.
Membre	Accéder à un profil particulier.
Membre	Contacter un autre membre via la page de profil, afin d'initier une conversation.
Membre	Changer mon avatar, gérer ma description, mes compétences, mes intérêts et mes disponibilités, afin de mettre à jour mon profil.
Membre	Consulter les disponibilités des autres membres.
Membre	Accéder à ma messagerie.
Membre	Envoyer un message.
Membre	Supprimer un message.
Membre	Modifier un message. <i>Absent du cadrage initial — fonctionnalité livrée au-delà du périmètre prévu</i> (PATCH /conversations/:id/message/:messageId , conv.router.ts:72-79).
Membre	Clore une conversation.
Membre	Suivre / ne plus suivre un membre.
Membre	Évaluer un autre membre à la fermeture d'une conversation.
Membre	Effectuer une recherche et filtrer par catégorie.
Membre	Me déconnecter.
Membre	Supprimer mon compte.

3.6. Règles de gestion arbitrées en cadrage

Plusieurs règles ont été tranchées collectivement : la mise en relation n'est ouverte qu'entre membres liés par un suivi ; l'évaluation ne devient disponible qu'à la clôture d'une conversation, chaque conversation portant un statut ; les disponibilités sont saisies par créneaux prédéfinis (jour × plage) plutôt que par calendrier libre. La fonctionnalité « souhaite apprendre » envisagée initialement a été retirée du périmètre.

3.7. Contraintes techniques et de compatibilité

Le cadrage fixait la pile technique et sa justification : Docker (cohérence des environnements), Node.js/Express (performances asynchrones, framework léger), Next.js (SEO via SSR/SSG), shadcn/ui + Tailwind (composants accessibles), PostgreSQL (base relationnelle robuste), Nginx (reverse proxy, SSL), Prisma (ORM type-safe), Meilisearch (recherche tolérante aux fautes), Socket.IO (temps réel). La compatibilité visée couvrait Chrome, Firefox, Edge et Safari, en desktop et mobile, dans leurs versions récentes.

Les choix de cette pile et leur mise en œuvre effective sont détaillés en section 7.

4. Présentation du commanditaire et de l'équipe

4.1. O'clock — école de développement à distance synchrone

Le projet a été réalisé dans le cadre de l'apothéose finale du Titre Professionnel Concepteur Développeur d'Applications (niveau 6, inscrit au RNCP), formation dispensée par O'clock — école de développement web et logiciel structurée autour d'un modèle pédagogique de téléprésentiel : les cours sont assurés en visio en temps réel par des formateurs. Notre promotion a été nommée Dublin.

4.2. L'apothéose — dispositif simulant l'entreprise

L'apothéose constitue la mise en situation professionnelle finale du cursus. Elle réunit un groupe d'apprenants pendant quatre semaines pour livrer un projet d'envergure depuis le cadrage jusqu'à la mise en production, en appliquant la méthodologie agile et l'ensemble des compétences techniques acquises. O'clock joue ici le rôle de commanditaire pédagogique : l'école définit le périmètre général — un projet web complet dont la fonctionnalité principale doit être d'envergure —, valide les choix techniques, et accompagne l'équipe via des points hebdomadaires avec un formateur référent.

4.3. L'équipe SkillSwap

Cinq développeurs apprenants, sur des rôles formalisés en sprint 0 et conservés tout au long du projet :

Membre	Rôle
Sébastien	Product Owner
Loïc	Scrum Master
Yorgan	Lead Back
Jérémy Soriano	Lead Front
Antoine	Frontend

La répartition détaillée des contributions, les rituels tenus et les outils de collaboration — Discord pour les sessions synchrones, Slack, Trello et un Drive partagé — sont décrits en section 5.

4.4. Le projet

L'équipe a porté SkillSwap, plateforme web d'échange de compétences entre particuliers. Le périmètre fonctionnel cible et les contraintes techniques structurantes sont détaillés au section 3. Le projet est aujourd'hui en production sous le nom de domaine skill-swap.fr.

5. Gestion de projet

5.1. Cadre et organisation de l'équipe

Le projet a été conduit par une équipe de cinq personnes en distanciel synchrone, avec des rôles distribués dès le sprint de cadrage : Sébastien (Product Owner), Loïc (Scrum Master), Yorgan (Lead Back), moi-même (Lead Front) et Antoine (Frontend). La coordination quotidienne s'est faite par salons vocaux Discord, avec des sessions de pair-programming ponctuelles.

5.2. Méthodologie : Scrum adapté à quatre semaines

Le projet s'est déroulé du 6 au 29 janvier 2026, soit quatre semaines découpées en quatre sprints d'une semaine. Le sprint 0 a été entièrement consacré au cadrage et à la conception ; les trois suivants au développement incrémental. Les rituels ont été adaptés au format distanciel : un point d'équipe chaque matin pour débriefer l'avancement et répartir la journée, puis une disponibilité continue en salon vocal permettant l'entraide et les décisions au fil de l'eau.

5.3. Le sprint 0 : cadrage et conception

La première semaine a produit l'ensemble des artefacts de conception avant toute ligne de code applicatif : présentation et clarification du cahier des charges, planification et attribution des rôles (lundi) ; user stories, ERD, MCD, dictionnaire de données et charte graphique (mardi) ; MPD, cas d'utilisation, diagramme de séquence, diagramme d'architecture et contrat d'endpoints (mercredi). La réalisation des maquettes m'a été attribuée ; Sébastien et Yorgan ont pris les endpoints et la matrice RBAC, Loïc l'initialisation Docker. La semaine s'est close par un débrief et la planification du sprint 1.

5.4. Déroulé des sprints

Sprint	Semaine	Commits	PR	Contenu principal
0	6–10 jan.	35	10	Cadrage, conception, initialisation
1	12–18 jan.	148	49	Authentification, landing page, Docker, pages profil et conversation
2	19–25 jan.	129	41	Follow/unfollow, édition de profil, recherche Meilisearch, Socket.IO
3	26–29 jan.	54	30	Messagerie temps réel, mise en production, documentation

Commits hors *merge* et Pull Requests mergées, comptés par semaine ISO sur le dépôt d'équipe. Le travail s'est poursuivi certains week-ends : les bornes de la colonne « Semaine » couvrent la semaine complète, week-end inclus.

5.5. Ma contribution et sa progression

J'ai porté six branches de feature et treize Pull Requests intégrées, en deuxième position sur le volume de commits de l'équipe. Ma contribution suit une montée en charge nette : quatrième contributeur au sprint 0 (3 commits, la semaine étant consacrée à la conception), je deviens deuxième au sprint 1 (41) puis premier contributeur au sprint 2 (54 commits). Le journal de bord tenu par l'équipe pendant le projet² retrace ce parcours : mise en place de l'architecture UI et des composants atomiques, landing page complète, configuration Docker de développement et de production, page profil, feature follow/unfollow, revue et refactorisation du frontend, puis conception et implémentation de la page de recherche en cohérence avec le backend Meilisearch.

5.6. Outils et flux de travail

Quatre outils ont structuré la collaboration : **Discord** pour les sessions synchrones et le pair-programming, **Slack** pour les échanges écrits, **Trello** pour le backlog³ et un **Drive partagé** pour les documents de cadrage — brief projet, tableur des user stories, journal de bord.

Le flux Git reposait sur une stratégie `main` / `dev` / branches de feature, avec intégration par Pull Request et branches courtes. La qualité de code était outillée par ESLint, Prettier et un hook de *pre-commit* Husky.

5.7. Anticipation des risques

Une analyse de risques a été conduite, identifiant douze risques répartis en cinq familles — techniques, développement, organisationnels, UX/sécurité et déploiement — chacun assorti d'une mitigation. J'ai par ailleurs formalisé une matrice de risques dans la documentation Arc42 du projet (délais, périmètre, performance de la recherche, sécurité des jetons), avec probabilité, impact, mitigation et plan de contingence.

²Tableur partagé, hors dépôt Git — comme les autres documents de cadrage (cahier des charges, user stories, dictionnaire de données). Le fichier `docs/carnet-de-bord.md` du dépôt est resté à l'état de gabarit.

³trello.com/b/536OvoxI/skillswap

6. Spécifications fonctionnelles

La présente section décline les choix fonctionnels du projet et leur traduction concrète : les contraintes ayant cadré la conception, l'architecture retenue, les maquettes d'interfaces, le modèle de données relationnel, le script SQL associé, et enfin la formalisation des cas d'utilisation — notamment celui qui sera approfondi techniquement en section 8.

Ces sept sous-sections forment un fil continu qui part de l'expression du besoin (6.1) jusqu'au comportement détaillé d'une fonctionnalité représentative (6.7), en passant par les différentes vues structurelles du système.

6.1. Contraintes du projet et livrables attendus

6.1.1. Contraintes

Le projet a été réalisé dans le cadre de l'apothéose finale de la formation O'clock — Promotion Dublin. Cette modalité simule un environnement professionnel et impose un ensemble de contraintes proches de celles d'une mission en entreprise.

Type de contrainte	Description
Temporelle	Apothéose conduite sur quatre semaines réparties en sprints courts (méthode Scrum), avec une livraison incrémentale et une démonstration en fin d'apothéose.
Humaine	Équipe de cinq personnes ⁴ avec rôles distribués (Lead Front, Lead Back, contributeurs full-stack), travaillant en distanciel synchrone via Discord (salons vocaux dédiés au pair-programming).
Technique — stack	Choix d'une stack moderne TypeScript de bout en bout : Next.js (App Router) côté front, Express côté back, PostgreSQL pour les données relationnelles, Meilisearch pour la recherche full-text, Socket.IO pour le temps réel. Choix documentés dans dix ADRs formalisés pendant le projet (cf. section 7.2).
Technique — qualité	Performance perçue inférieure à 500 ms (mesure Lighthouse), couverture OWASP Top 10 sur la sécurité, conformité visée RGAA 4.1 / WCAG 2.1 AA ⁵ , conventions de code partagées (Prettier, ESLint).
Réglementaire	Conformité RGPD : minimisation des données collectées, consentement explicite, droits d'accès et de suppression.

⁴Composition de l'équipe Dublin : Sébastien (Product Owner), Loïc (Scrum Master), Yorgan (Lead Back), Jérémy Soriano (Lead Front) et Antoine (Frontend).

⁵L'audit accessibilité formel via `axe-core` et Lighthouse est planifié mais non réalisé à la date de rédaction. Cette dette qualité est documentée en section 13.

	Mentions légales et politique de confidentialité requises avant mise en production.
Fonctionnelle	Périmètre MVP imposé : authentification, profil et compétences, recherche de membres, messagerie temps réel, système de notation et suivi entre membres. Hors-scope : paiement, application mobile native, vidéoconférence, géolocalisation avancée.

6.1.2. Livrables attendus

L'apothéose impose une chaîne complète de livrables, depuis la conception jusqu'à la mise en production effective :

Livable	Description	État
Application en production	Plateforme accessible publiquement à skill-swap.fr , intégrant l'ensemble des fonctionnalités MVP.	✓ Livré
Base de données	Schéma PostgreSQL en six migrations Prisma successives, avec seed de développement peuplant 41 utilisateurs de démonstration ; le seed de production n'initialise que les référentiels (rôle, catégories, compétences, créneaux).	✓ Livré
Documentation technique	Livable d'équipe mergé sur <code>main</code> : stratégie de documentation (<code>docs/documentation-strategy/</code>), diagrammes UML (<code>docs/uml/</code>) et référence des endpoints (<code>docs/endpoints/</code>). S'y ajoutent, produits pendant le projet sur la branche <code>Documentation</code> mais jamais mergés sur <code>main</code> , les dix ADRs et une spécification OpenAPI 3.0 déclarant les 37 endpoints (21 janvier 2026). Le guide utilisateur Diátaxis et la mise en ligne de l'ensemble sur Vercel sont, eux, une extension post-projet.	✓ Livré ⁶
Tests automatisés	Tests d'intégration backend uniquement : sept fichiers <code>*.spec.test.ts</code> écrits pour le Node Test Runner natif, visant les contrôleurs et les events Socket.IO. Cinq des sept suites échouent sur un bug de fixture identifié et documenté — la donnée de rôle n'est pas créée avant les utilisateurs de test. Dette assumée, non corrigée avant la soutenance.	✓ Partiel ⁷

⁶Le statut « Livré » porte sur la documentation d'équipe et sur les artefacts produits pendant le projet. La publication en ligne et le guide Diátaxis sont postérieurs à la soutenance.

⁷Deux limites cumulées : cinq des sept suites backend en échec sur fixture, et aucun test automatisé côté frontend dans le livrable (ni unitaire, ni E2E, ni catalogue de composants). Couverture jamais mesurée. Cf. section 10.

Conteneurisation	Stack Docker Compose à six services en production (frontend, backend, postgres, meilisearch, nginx, certbot) avec configurations dev et prod distinctes.	✓ Livré
Pipeline de déploiement continu (CD)	Déploiement automatisé vers le VPS de production sur push <code>main</code> (<code>.github/workflows/deploy-prod.yml</code>) : connexion SSH, mise à jour du dépôt, reconstruction et redémarrage des conteneurs. Aucune étape d'intégration continue — le workflow ne lance ni test, ni lint, ni build de vérification avant de déployer.	✓ Livré

6.2. Architecture logicielle du projet

L'architecture retenue est celle d'un **monolithe modulaire conteneurisé** : les responsabilités sont strictement séparées entre frontend et backend (deux applications distinctes, deux conteneurs Docker, deux processus de build), mais le backend reste un seul processus Node.js qui sert à la fois les routes REST et le serveur Socket.IO. Cette approche évite la complexité opérationnelle des microservices tout en autorisant un découpage clair des concerns.

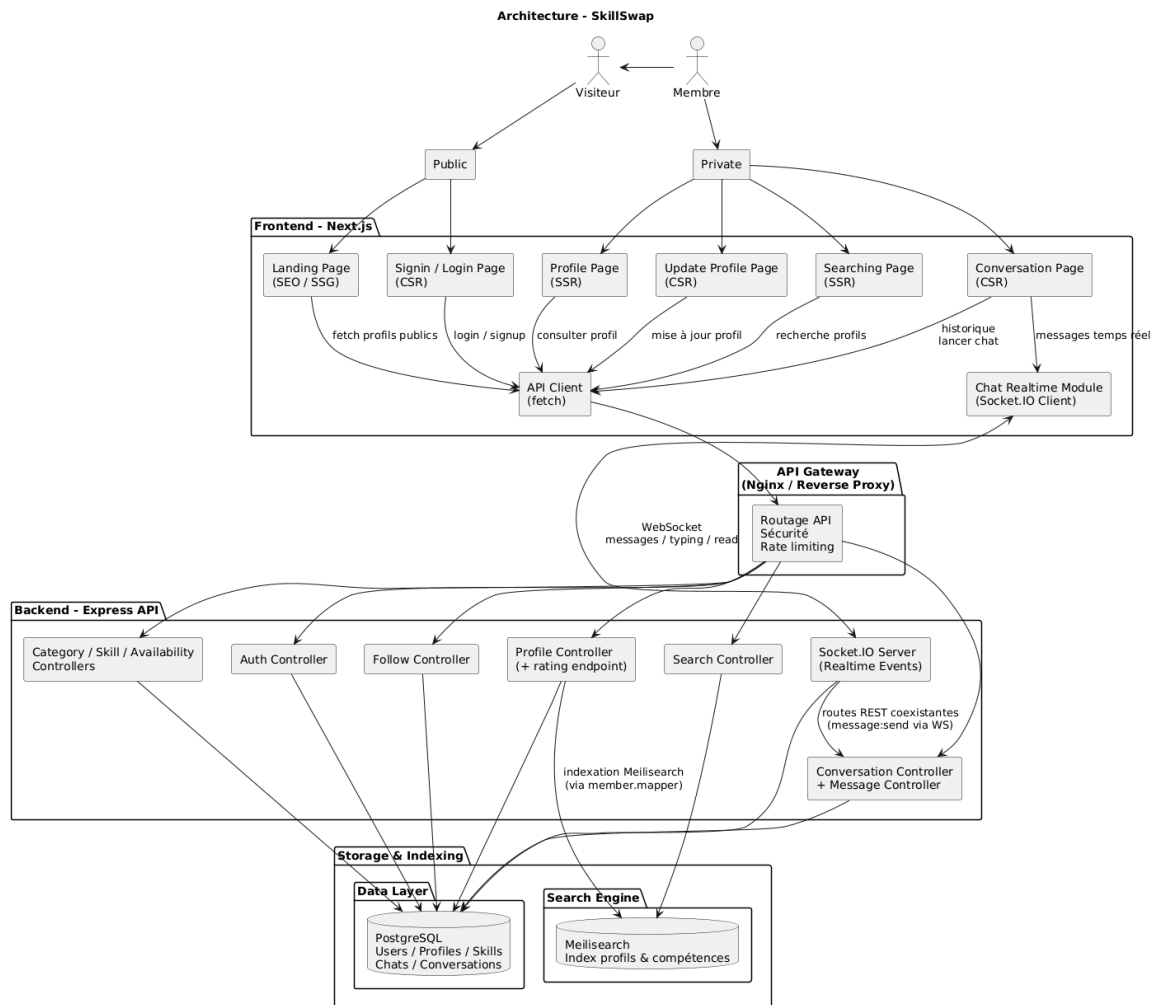


Fig. 1. – Architecture logicielle macro de SkillSwap. Le proxy Nginx assure la terminaison TLS et le routage des requêtes vers le frontend Next.js et l'API Express. Le serveur Express héberge à la fois les routes REST (`/api/v1/*`) et le serveur Socket.IO (sur le même port). PostgreSQL persiste les données relationnelles ; Meilisearch sert l'index de recherche full-text, alimenté par une réindexation lancée manuellement (script `search:reindex`) — il n'y a pas de réindexation automatique au démarrage du serveur.

Provenance des figures UML. Les sources PlantUML sont des livrables d'équipe (`docs/uml/**.puml`). Quatre figures de cette section — architecture (ci-dessus), parcours utilisateur, ERD et cas d'utilisation — sont des **rendus PNG régénérés pour ce dossier** à partir de ces mêmes sources, via PlantUML. Deux d'entre elles portent un autre nom dans le livrable : `diagramme-architecture.png` et `use-case.png`. Les figures `arborescence.png` et `sequence/conversation.png` sont, elles, reprises telles quelles du livrable.

6.2.1. Communications inter-composants

Le frontend communique avec le backend via deux canaux complémentaires : **REST** (`/api/v1/*`) pour les requêtes synchrones (authentification, CRUD, listing initial des messages) et **WebSocket** via Socket.IO sur le même serveur HTTP pour les flux temps

réel (envoi/réception de messages, notifications de nouvelles conversations). Ce choix d'architecture hybride est documenté dans l'ADR-011 — formalisé a posteriori pour ce dossier — et détaillé techniquement en sous-section 8.4.

6.3. Maquettes et enchaînement des maquettes

Les maquettes ont été produites en amont du développement à l'aide de **Figma**, sur la base d'un atelier de cadrage produit avec l'équipe au début de l'apothéose. Elles définissent la charte graphique, la hiérarchie des écrans et les principaux états interactifs.

6.3.1. Arborescence de l'application

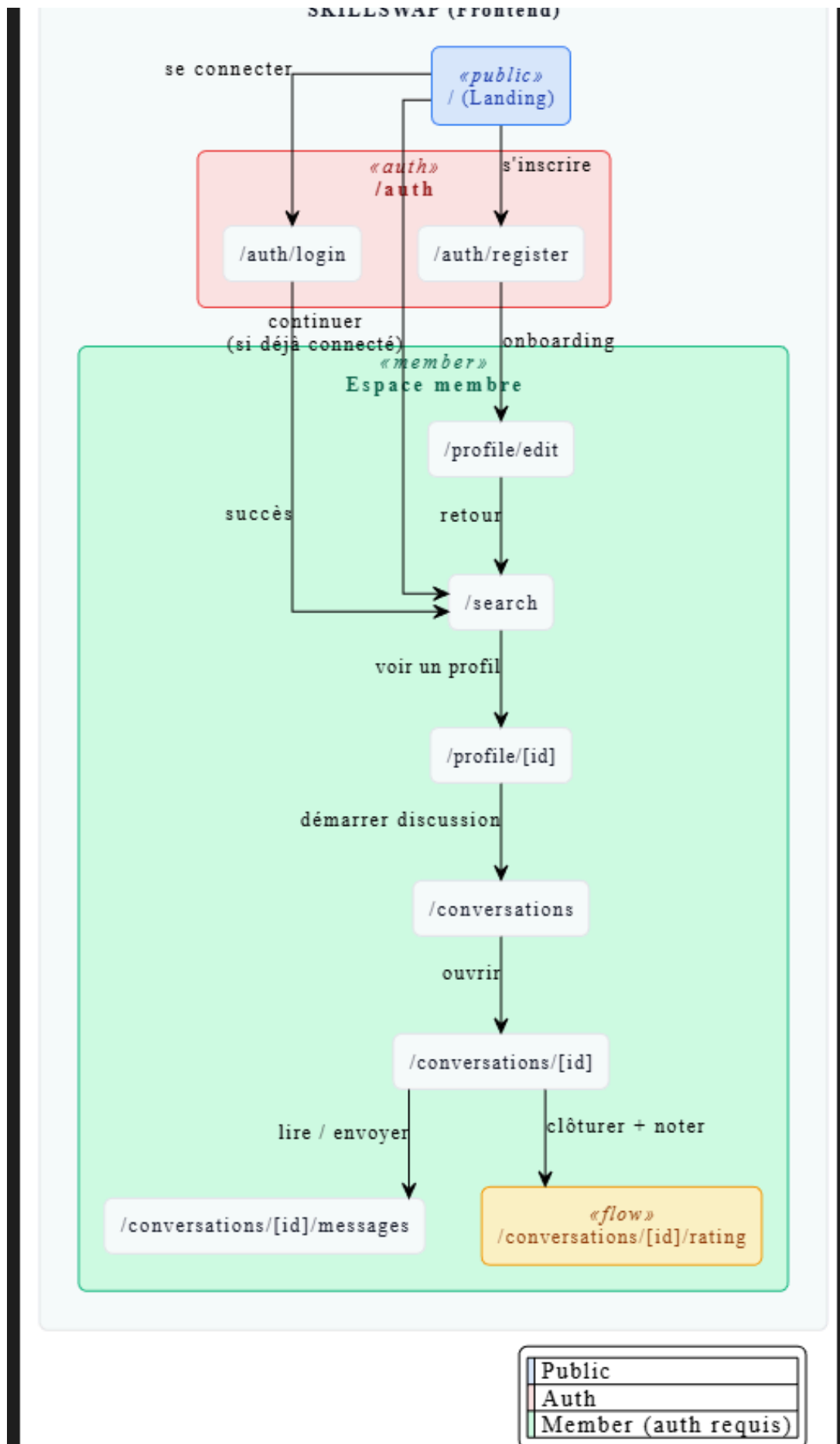


Fig. 2. – Arborescence des écrans publics et privés de SkillSwap. La racine `/` présente la page d'accueil (catégories, membres, lieux notés). Les écrans en zone publique (consultation profil, page CGU, mentions légales) sont accessibles sans authentification ; les écrans en zone privée (édition profil, recherche, conversations) nécessitent une session valide.

L'arborescence reflète le principe de **progressive disclosure** : la zone publique reste librement accessible, et l'authentification n'est requise qu'au moment d'une action transactionnelle (envoi de message, ajout de compétence, suivi de membre).

6.3.2. Parcours utilisateur principal

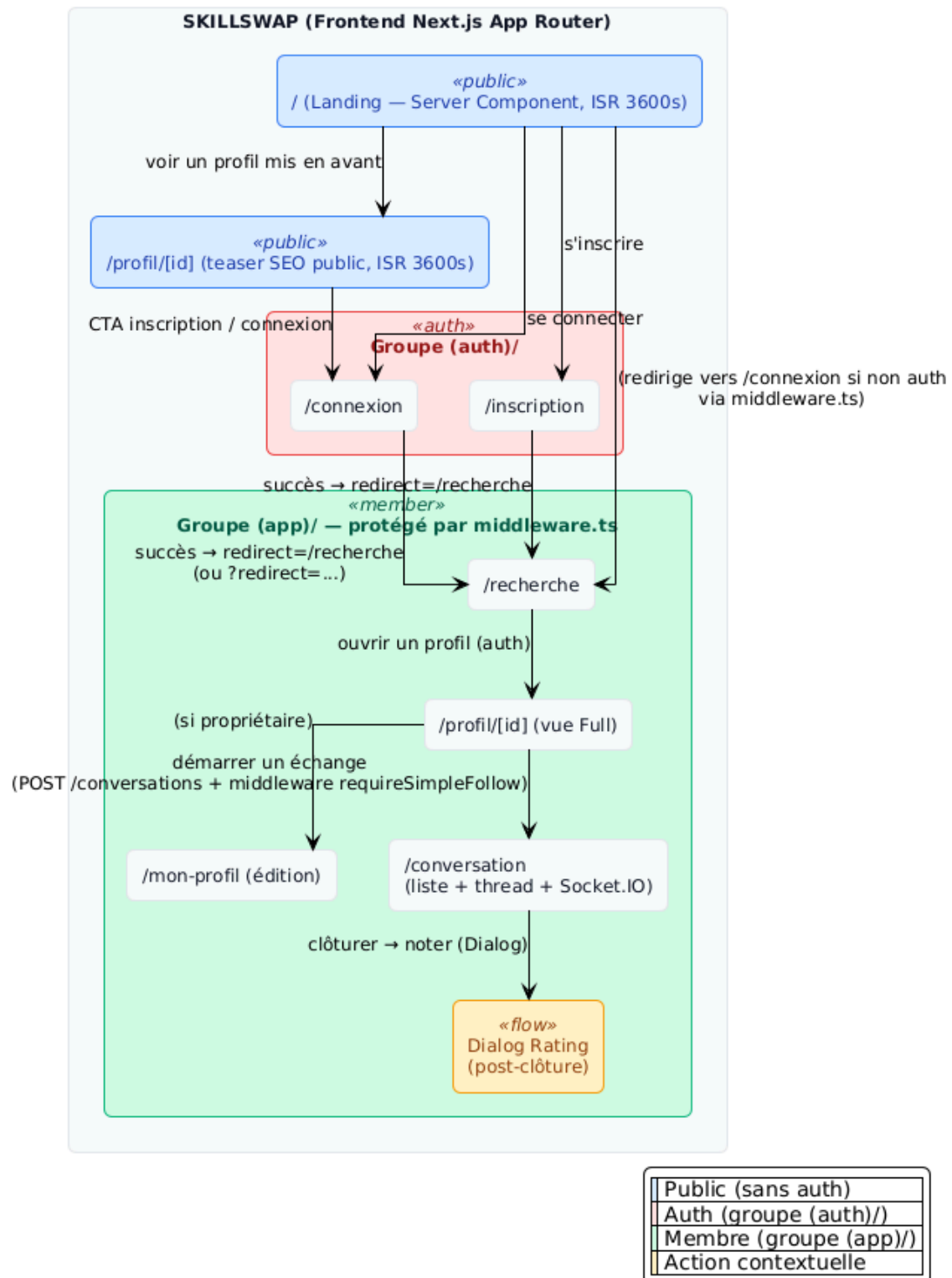
SkillSwap — UserFlow (Frontend)

Fig. 3. – Parcours utilisateur d'un nouveau membre — inscription → création du profil → recherche → suivi → ouverture de la première conversation. Cette séquence est exactement celle simulée par le jeu d'essai en section 11.

6.3.3. Captures de maquettes Figma

Capture à insérer : ../assets/captures-ui/05-figma-overview.png

Fig. 4. – Vue d’ensemble du board Figma — atomes, composants et maquettes assemblées des principaux écrans (accueil, profil, recherche, messagerie).

6.4. Modèle entités-associations et modèle physique de la base de données

La modélisation suit les trois niveaux Merise. Le **MCD** a été formalisé en notation Mocodo à partir des règles de gestion ; le **MLD** et le **MPD** ont été reconstruits de façon déterministe depuis une base PostgreSQL montée à partir des six migrations réelles, en interrogeant le catalogue `information_schema` — aucune inférence depuis le code applicatif. Le fichier `schema.prisma` reste la source de vérité opérationnelle qui génère les migrations, ce qui garantit la cohérence stricte entre le schéma documenté et le schéma physique appliqué en production.

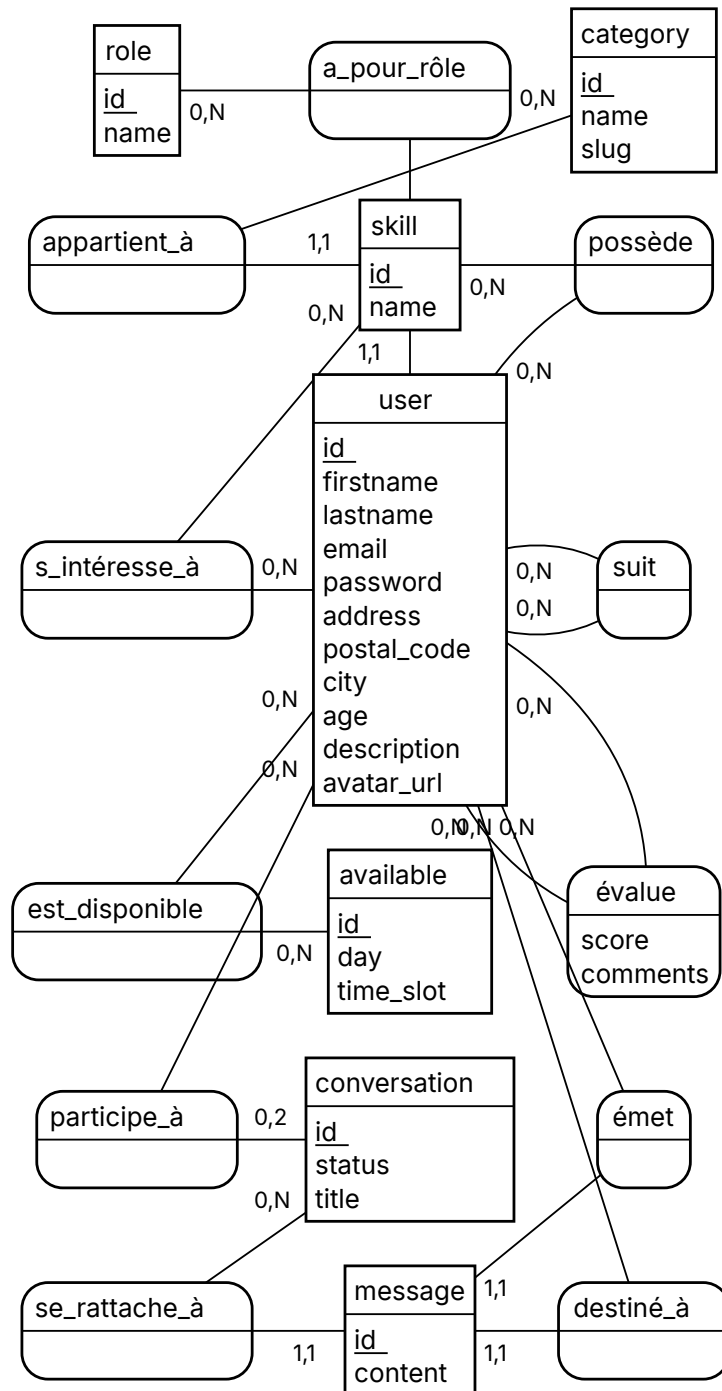


Fig. 5. – Modèle conceptuel de données (Merise, notation Mocodo) — sept entités métier et onze associations, dont une association porteuse `évalue` (attributs `score` et `comments`). L'entité technique `refresh_token` est volontairement exclue du niveau conceptuel.

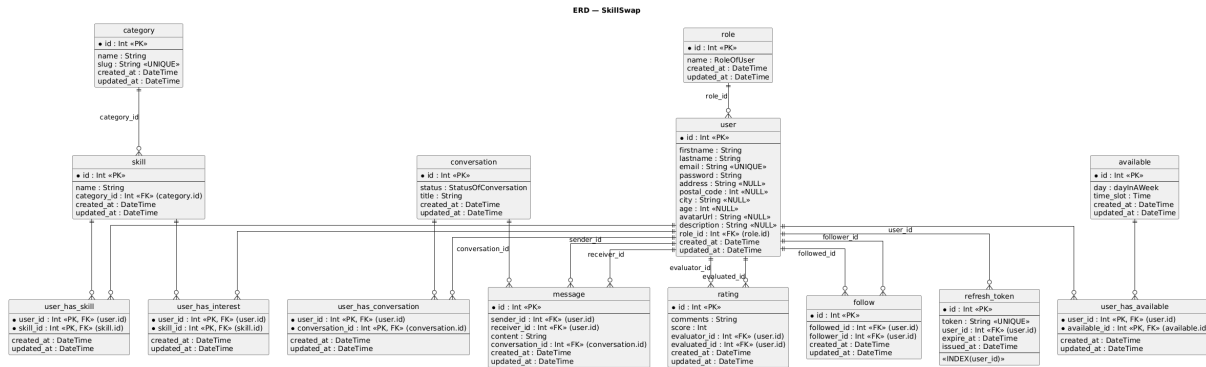


Fig. 6. – Diagramme entité-relation complet de SkillSwap — quatorze modèles, regroupés en cinq domaines fonctionnels : identité, compétences, disponibilités, échange, social.

6.4.1. Domaines fonctionnels du modèle

Les quatorze modèles se regroupent en cinq domaines cohérents :

Domaine	Modèles
Identité	<code>User</code> , <code>Role</code> , <code>RefreshToken</code> — gestion des comptes, des rôles et des sessions par rotation de tokens.
Compétences	<code>Skill</code> , <code>Category</code> , <code>UserHasSkill</code> , <code>UserHasInterest</code> — référentiel des compétences disponibles sur la plateforme et expression des compétences offertes / recherchées par chaque membre.
Disponibilités	<code>Available</code> , <code>UserHasAvailable</code> — créneaux horaires standardisés associés à chaque membre.
Échange	<code>Conversation</code> , <code>Message</code> , <code>UserHasConversation</code> — coeur de l'échange, traité en détail en section 8.
Social	<code>Follow</code> , <code>Rating</code> — graphe de suivi entre membres et système de notation.

La différence de comptage entre les deux niveaux est normale et voulue : le MCD recense **sept entités métier** (`user`, `role`, `category`, `skill`, `available`, `conversation`, `message`), tandis que le MLD en compte **quatorze** après transformation. Les sept tables supplémentaires sont la matérialisation des associations : les quatre tables de jonction issues des relations N-N, les deux associations réflexives `follow` et `evaluation`, et l'entité technique `refresh_token`, exclue du niveau conceptuel parce qu'elle relève de l'authentification et non du métier.

Les contraintes d'unicité critiques sont matérialisées au niveau base :

`@@unique([followedId, followerId])` sur `Follow` (graphe orienté, non-auto-suivi appliqué côté middleware) et `@@unique([evaluatorId, evaluatedId])` sur `Rating` (mappée `evaluation` en BDD), qui empêche tout doublon de notation entre une même paire de membres.

6.5. Script de création et de modification de la base de données

L'évolution du schéma est gérée par six migrations Prisma successives, chacune représentant une étape datée de la conception. La traçabilité SQL de cette évolution est intégrale et auditable.

Date	Migration	Objet
2026-01-12	<code>init_db</code>	Création initiale du schéma : 13 tables, contraintes de clés étrangères, index primaires, enums <code>RoleOfUser</code> , <code>StatusOfConversation</code> et <code>dayInAWeek</code> .
2026-01-14	<code>add_category_slug</code>	Ajout du champ <code>slug</code> sur <code>Category</code> pour les URLs SEO-friendly des pages catégorie.
2026-01-16	<code>create_relation_table_user_available</code>	Refonte du modèle de disponibilités : suppression des colonnes <code>start</code> / <code>end</code> / <code>user_id</code> de <code>available</code> , introduction de l'enum <code>Time</code> (<code>Morning</code> / <code>Afternoon</code>) et création de la table de jonction N-N <code>user_has_available</code>

2026-01-17	<code>fix_snake_case</code>	Renommage de la colonne <code>avatarUrl</code> → <code>avatar_url</code> sur la table <code>user</code> (réalignement camelCase → snake_case oublié à la migration initiale).
2026-01-18	<code>add_unique_constraint</code>	Ajout de contraintes d'unicité manquantes (<code>follow</code> sur <code>(followed_id, follower_id)</code> , <code>evaluation</code> sur <code>(evaluator_id, evaluated_id)</code>).
2026-01-20	<code>make_the_comment_field_in_the_rating_table_optional</code>	Passage du champ <code>comments</code> de <code>evaluation</code> en facultatif (UX : l'évaluateur peut donner une note sans être obligé de commenter).

L'extrait ci-dessous est issu du schéma physique réel, obtenu par `pg_dump --schema-only` sur une base montée à partir des six migrations — il reflète donc l'état effectivement appliqué, et non une reconstitution :

```
CREATE TYPE public."RoleOfUser" AS ENUM (
    'Membre'
);

CREATE TABLE public."user" (
    id integer NOT NULL,
    firstname text NOT NULL,
```

```

    lastname text NOT NULL,
    email text NOT NULL,
    password text NOT NULL,
    address text,
    postal_code integer,
    city text,
    age integer,
    description text,
    role_id integer NOT NULL,
    created_at timestamp(3) without time zone DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_at timestamp(3) without time zone DEFAULT CURRENT_TIMESTAMP NOT NULL,
    avatar_url text
  );

```

L'ordre des colonnes porte la trace des migrations successives : `avatar_url` figure en dernière position parce qu'elle a été recrée par la migration `fix_snake_case`, postérieure à la création initiale de la table.

Les extraits SQL générés par Prisma — création de la table `user` dans `init_db` et matérialisation des contraintes d'unicité dans `add_unique_constraint` — sont reproduits intégralement en **annexe A**. Prisma génère le SQL au format conventionnel : type `TEXT` sans contrainte de longueur (les bornes sont appliquées en amont par les schémas Zod côté serveur), unicités via `CREATE UNIQUE INDEX` séparés, et FK déclarées en fin de migration via `ALTER TABLE`. Les contraintes d'unicité de `Follow` et `evaluation` sont appliquées au niveau base — défense en profondeur garantissant l'intégrité même en cas de bug applicatif.

6.6. Diagramme du comportement des fonctionnalités — cas d'utilisation

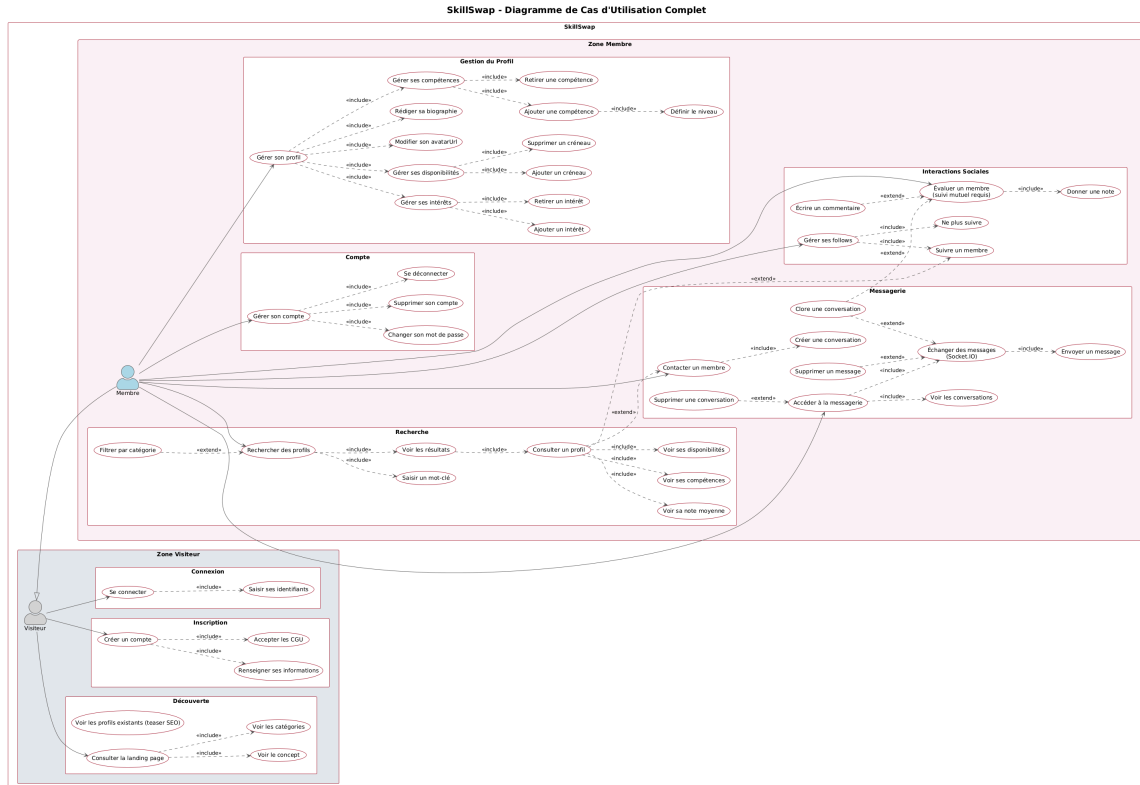


Fig. 7. – Diagramme de cas d'utilisation de SkillSwap. Deux acteurs : le visiteur non-authentifié et le membre authentifié. Les cas d'utilisation grisés sont hors-scope MVP et reportés à la V2.

6.6.1. Acteurs et cas d'utilisation principaux

Visiteur non-authentifié — peut consulter les profils publics, parcourir les catégories, lire les mentions légales et la politique de confidentialité, et bien sûr s'inscrire ou se connecter. Aucune action transactionnelle (message, suivi, évaluation) ne lui est accessible.

Membre authentifié — accède à l'ensemble des fonctionnalités MVP : édition complète de son profil (compétences, intérêts, disponibilités, avatar), recherche d'autres membres avec filtre par catégorie, suivi / désuivi de membres, ouverture de conversations avec les membres suivis, envoi et clôture de conversations, évaluation post-conversation des membres avec qui il a échangé.

Aucun rôle d'administration n'est implémenté. La table `role` existe et chaque utilisateur y est rattaché par une clé étrangère `role_id` non nulle, mais l'énumération `RoleOfUser` ne contient qu'une seule valeur, `Membre` — vérifiable en base par `SELECT enumLabel FROM pg_enum`. Le `roleId` est bien embarqué dans le JWT et exposé sur la requête par le middleware `checkAuth`, mais il n'est lu par aucun middleware ni contrôleur pour prendre une décision d'autorisation.

La table `role` constitue donc un **point d'extension du modèle de données**, prêt à accueillir d'autres valeurs, et non un mécanisme actif : la modération est une dette

assumée, reportée en V2. L'autorisation réellement en vigueur en V1 est binaire (visiteur / membre) et repose sur trois mécanismes indépendants du rôle : l'authentification (`checkAuth`), la propriété de la ressource (`isOwner`) et la relation sociale (`requireFollow`).

Le périmètre MVP couvre ainsi l'essentiel des cas d'utilisation visiteur et membre, l'effort de la promotion ayant été concentré sur le coeur produit.

6.7. Diagramme du détail du cas d'utilisation le plus significatif

Le cas d'utilisation le plus significatif retenu est **l'envoi d'un message à un membre suivi** : il combine l'ensemble des prérequis fonctionnels de la plateforme (authentification, suivi, conversation), mobilise les deux canaux de communication (REST pour la création de conversation, Socket.IO pour l'envoi et la réception), et illustre les patterns techniques les plus défendables du projet (optimistic UI, cloisonnement par rooms, gating métier). Ce choix résulte d'un audit comparatif documenté qui l'identifie comme la seule fonctionnalité couvrant exhaustivement les axes du référentiel⁸.

⁸Audit interne : `docs/audits/feature-inventory-cda.md`. Ce cas d'utilisation est repris en section 8 sous l'angle technique fin et en section 11 sous l'angle comportemental observable.

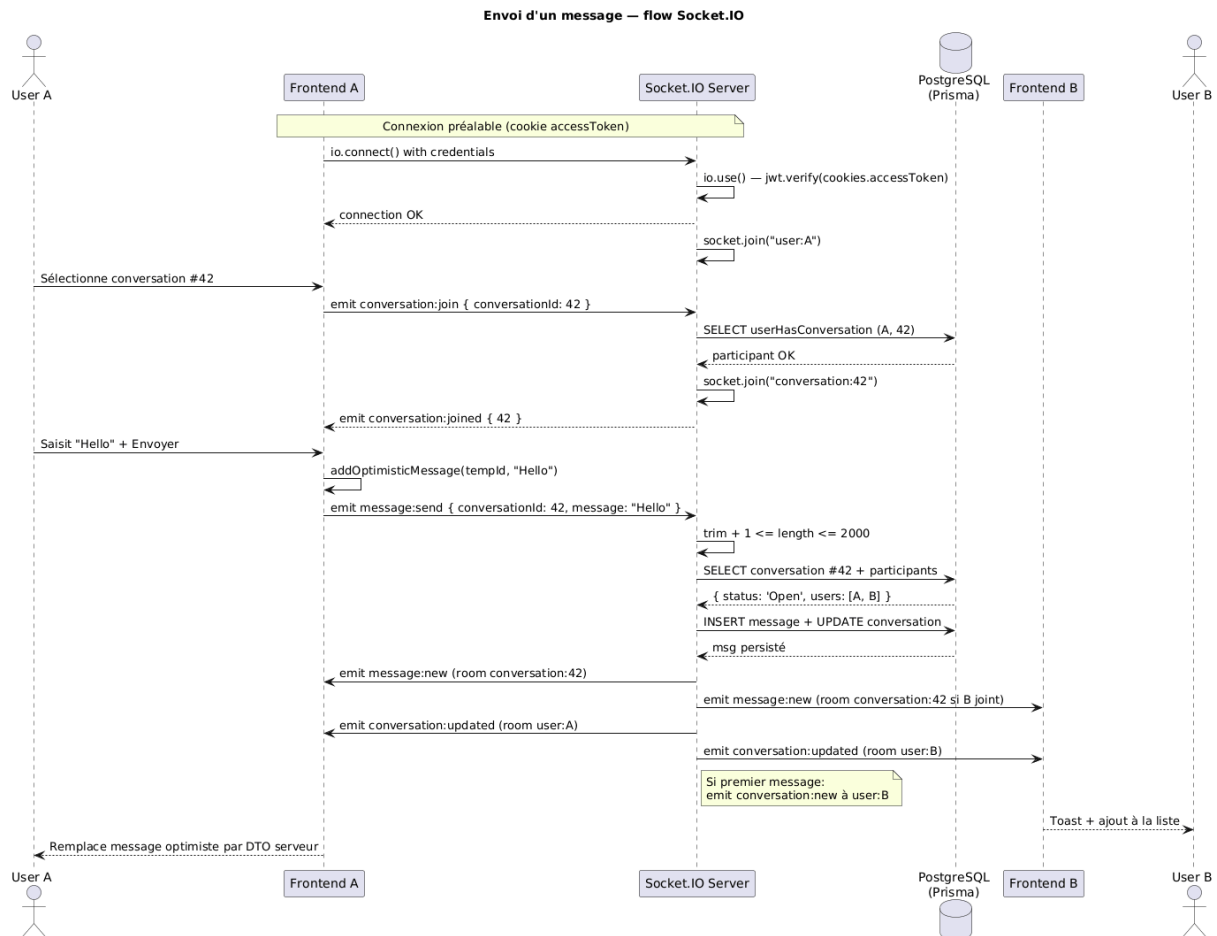


Fig. 8. – Diagramme de séquence — envoi d'un message d'Alice vers Bob. Six lignes de vie : Alice et son frontend, le serveur Socket.IO, la base PostgreSQL accédée via Prisma, le frontend de Bob et Bob. Les flèches en pointillés représentent les retours synchrones ; les flèches pleines, les events asynchrones poussés par le serveur.

6.7.1. Lecture du diagramme

Le diagramme illustre le pipeline complet en huit étapes :

1. **Connexion préalable** : à l'authentification, le client Socket.IO se connecte avec le cookie `accessToken` ; le serveur vérifie le JWT et inscrit la socket dans la room personnelle `user:Alice`.
2. **Sélection d'une conversation** : Alice sélectionne la conversation 42 dans sa liste. Le frontend émet `conversation:join` ; le serveur valide qu'Alice est bien participante de cette conversation et l'inscrit dans la room `conversation:42`.
3. **Saisie et envoi optimistic** : Alice rédige un message et clique « Envoyer ». Le frontend ajoute immédiatement le message dans la liste locale avec un identifiant temporaire négatif, puis émet `message:send` au serveur sans attendre.
4. **Validation serveur** : le serveur vérifie la longueur du message (1 à 2000 caractères), le statut de la conversation (refusé si `close`), et la qualité de participant de l'émetteur.
5. **Persistance** : un `Promise.all` Prisma crée le message en table `message` et met à jour le `updated_at` de la `conversation` parent.

6. **Diffusion ciblée** : le serveur émet `message:new` à la room `conversation:42` (Alice et Bob s'ils sont tous deux dans la room), et `conversation:updated` aux rooms `user:Alice` et `user:Bob`.
7. **Cas particulier — premier message** : si c'est le premier message de la conversation, le serveur émet en plus `conversation:new` à la room `user:Bob`, déclenchant un toast d'apparition côté Bob.
8. **Réconciliation côté Alice** : Alice reçoit `message:new` ; le filtre côté hook ignore le retour serveur lorsque `sender.id === user.id` (l'optimistic local reste en place comme affichage final).

Le détail technique de chaque étape — code des handlers, structure des events, gestion d'erreurs — est traité en sous-section 8.4 ; le présent diagramme se contente d'en donner la vue comportementale macro.

7. Spécifications techniques

La présente section couvre les choix techniques transversaux du projet — stack, architecture, patterns et sécurité de niveau infrastructure. Les déclinaisons fonctionnelles spécifiques à la messagerie (auth Socket, gating métier, cloisonnement par rooms) sont traitées en section 8.

7.1. Stack technique

Le projet adopte une stack **TypeScript de bout en bout**, de la base de données au navigateur, ce qui réduit la surface d'erreurs d'intégration et autorise une seule boîte à outils mentale pour toute l'équipe.

Composant	Technologie	Version
Frontend	Next.js (App Router) + TypeScript + Tailwind CSS + shadcn/ui	16.1.1
Backend API	Express + TypeScript + Zod	5.2.1
ORM	Prisma Client	7.2.0
Base de données	PostgreSQL	16
Recherche full-text	Meilisearch	server 1.6 / client 0.55.0
Temps réel	Socket.IO (intégré au serveur HTTP Express)	4.8.3
Infrastructure	Docker Compose + Nginx (reverse proxy + TLS)	—
Tests backend	Node Test Runner natif (sans framework tiers)	—
Tests frontend	Aucun outil de test dans le livrable (Vitest et Playwright planifiés — cf. ADR-010)	—

7.2. Choix architecturaux — synthèse des ADRs

Dix décisions d'architecture ont été formalisées en ADRs (Architecture Decision Records) pendant le projet. La décision d'adopter Socket.IO pour le temps réel a été formalisée a posteriori dans un onzième ADR rédigé pour ce dossier⁹. Synthèse :

N°	Sujet	Décision retenue
----	-------	------------------

⁹Les dix ADRs du projet — `001-nextjs.md` à `010-testing-strategy.md` — se trouvent dans `docs/documentation-implementation/arc42/09-decisions/` du dépôt d'équipe (branche `Documentation`). Le onzième, `011-socket-io.md`, a été rédigé le 8 mai 2026 pour ce dossier : il formalise après coup un choix technique réellement en production depuis janvier 2026. Chaque ADR documente le contexte, les options envisagées, la décision retenue et ses conséquences.

001	Framework frontend	Next.js (App Router) — SSR pour le SEO des profils publics, écosystème React maîtrisé, configuration de production simple.
002	Styling frontend	Tailwind CSS + shadcn/ui — utility-first, cohérence visuelle, composants accessibles par défaut.
003	Accès aux données	Prisma — ORM type-safe, migrations versionnées, génération automatique du client TypeScript.
004	Gestion d'état serveur	TanStack Query avait été envisagé (ADR-004) mais n'a finalement pas été intégré ; la gestion des données côté client s'appuie sur une composition de hooks React natifs.
005	Validation des entrées	Zod — schémas type-safe côté serveur et côté client, volontairement dupliqués faute de package partagé ; messages d'erreur explicites via un middleware Express maison de six lignes.
006	Architecture des composants UI	Atomic Design — atoms / molécules / organismes / pages, lisibilité accrue.
007	Authentification	JWT + cookies httpOnly + refresh token rotatif — pas d'exposition côté client, rotation automatique à chaque refresh.
008	Recherche full-text	Meilisearch — performance et simplicité d'API supérieures à la recherche full-text native de PostgreSQL, ressources opérationnelles maîtrisées.
009	Stratégie de bascule	Mock → API — développement front sur mocks réalistes en début d'apothéose, puis migration progressive vers l'API réelle jusqu'au retrait complet des mocks.
010	Stratégie de tests	Pyramide diversifiée décidée, chaque outil sur son étage : TypeScript (types), Storybook (composants UI), Vitest (hooks et utilitaires), Playwright (E2E), TypeDoc pour la documentation d'API. Hors l'étage TypeScript, acquis de fait, aucun de ces quatre outils n'a été intégré au livrable ; les tests effectivement écrits sont côté backend — sept specs d'intégration au Node Test Runner natif.
011	Communications temps réel	Socket.IO — auth par cookie partagé, modèle de rooms (<code>user:X</code> + <code>conversation:Y</code>), serveur unique avec l'API REST.

7.3. Patterns transversaux

Le code est structuré selon trois patterns récurrents qui assurent homogénéité et testabilité.

Backend en couches. Chaque domaine fonctionnel est décliné dans la même séquence : un **router** expose les endpoints, des **middlewares** appliquent l'authentification (`checkAuth`) et les règles métier (`requireSimpleFollow`, `isOwner`, `parseNumericParams`), un **controller** traduit la requête HTTP en appel de service, le **service** contient la logique métier et appelle Prisma. Cette régularité permet de localiser n'importe quelle fonctionnalité backend par convention plutôt que par documentation.

Le sens des dépendances a été vérifié mécaniquement avec `dependency-cruiser` — 72 modules et 169 dépendances analysés : aucun *controller* ni *router* n'importe Prisma, et aucun service ne dépend de la couche présentation — zéro violation. Cinq modules font néanmoins exception en accédant au client Prisma hors de la couche service : `realtime/socket.ts` (persistance des messages temps réel sans repasser par les services REST), `middlewares/conv.middleware.ts` (la vérification du lien de suivi précède le controller), `middlewares/error.middleware.ts` (import de types seul, aucune requête), `mappers/member.mapper.ts` (projection vers Meilisearch) et `lib/auth.ts` (écriture du refresh token à l'émission). Ces cinq points sont la dette d'architecture identifiée du backend.

Atomic Design côté frontend. Les composants sont rangés en quatre niveaux — atoms (boutons, inputs, badges issus de `shadcn/ui`), molécules (items composés réutilisables), organismes (sections de page), et pages (routes Next.js). Le périmètre messagerie illustre cette discipline : neuf organismes dans `ConversationPage/` composent les molécules `MessageBubble`, `ConversationItem` et les atoms `Button` / `Input`.

Type-safety end-to-end. Les types TypeScript se propagent de la base de données (générés par Prisma) jusqu'à l'UI (consommés par les hooks React). La validation des entrées s'appuie sur Zod des deux côtés, mais les schémas ne sont **pas** mutualisés : cinq schémas serveur (`backend/src/validation/`) et quatre schémas client (`frontend/src/lib/validation/`) coexistent en duplication assumée. Extraire un package partagé imposait un build TypeScript commun et une publication interne, effort écarté avant la soutenance ; la cohérence contractuelle repose donc sur une discipline de relecture, et constitue la dette technique la plus visible du projet.

7.4. Sécurité transversale

Cinq mécanismes de sécurité couvrent l'ensemble des routes et des échanges, complétés par les contrôles spécifiques à la messagerie détaillés en section 8.

Domaine	Mesure et implémentation
---------	--------------------------

Hashing mots de passe	<code>argon2</code> (paramètres par défaut de la lib <code>argon2</code> Node) appliqué dans <code>auth.service.ts:21</code> avant insertion en base. Pas de hashage côté client.
Sessions	JWT signés (<code>jsonwebtoken</code>) en cookies <code>httpOnly + secure + sameSite='strict'</code> en production (<code>auth.controller.ts:63-94</code>). Refresh token rotatif renouvelé à chaque appel <code>/api/v1/auth/refresh</code> (<code>auth.service.ts:103-108</code>), invalidation de l'ancien.
Validation des entrées	Schémas Zod appliqués par middleware déclaratif sur 18 des 37 routes applicatives (<code>body</code> , <code>query</code> , <code>params</code>) — register, login, profil, conversations, recherche, catégories. Les routeurs <code>follow</code> , <code>skill</code> et <code>availability</code> n'ont pas de schéma : <code>follow</code> ne valide que des identifiants numériques, convertis par <code>parseNumericParams</code> (<code>auth.middleware.ts:38-45</code>), tandis que <code>skill</code> et <code>availability</code> n'exposent qu'une lecture sans paramètre — écart assumé. En cas d'échec, <code>parseAsync</code> lève et le gestionnaire d'erreurs répond <code>422 Unprocessable Entity</code> avec la liste des messages de validation (<code>error.middleware.ts:32-33</code>).
Transport	HTTPS forcé en production via Nginx + certificats Let's Encrypt renouvelés automatiquement. Redirection HTTP → HTTPS systématique.
CORS	Une origine autorisée, configurée par variable d'environnement, appliquée à Express (<code>app.ts:12-17</code>) comme au handshake Socket.IO (<code>socket.ts:81-82</code>) ; rejet des requêtes hors-domaine. En production, le serveur refuse de démarrer si <code>ALLOWED_ORIGIN</code> est absente (<code>config.ts:29-38</code>).

Quatre zones de fragilité sont assumées et reportées en V2 : **Helmet installé mais non monté côté Express** (la dépendance `helmet@8.1.0` est présente dans `backend/package.json` mais n'est jamais appliquée — les en-têtes de sécurité courants `X-Frame-Options`, `X-Content-Type-Options`, `X-XSS-Protection` et HSTS sont néanmoins posés par Nginx en façade en production, ce qui couvre l'essentiel) ; **Content Security Policy stricte absente** (ni côté Express, ni côté Nginx) ; **audit accessibilité formel non réalisé** (RGAA 4.1 / WCAG 2.1 AA visés mais non mesurés via `axe-core` ou Lighthouse) ; **rate-limiting absent** (aucune protection applicative contre l'abus en volume — ni `express-rate-limit`, ni `limit` côté Nginx). Ces dettes ne remettent pas en cause la stabilité observée en production mais constituent les premiers chantiers de durcissement à programmer.

8. Réalisations — Messagerie temps réel

La présente section porte sur la **fonctionnalité la plus représentative** du projet, au sens du référentiel : la messagerie temps réel entre membres. Ce choix n'est pas anodin — il résulte d'un audit fonctionnel comparatif que j'ai conduit pour la préparation de ce dossier¹⁰, et qui a évalué neuf fonctionnalités principales du projet selon cinq critères (cœur métier, couverture technique, visibilité du travail Lead Front, enjeux de sécurité, richesse pour la démonstration). La messagerie obtient un score de 25 sur 25, seule à couvrir l'ensemble des axes en un périmètre unique.

Plus fondamentalement, la messagerie est le moment où SkillSwap cesse d'être un annuaire de profils et devient une plateforme d'échange. Sans elle, deux membres aux compétences complémentaires ne peuvent jamais entrer en relation. Le code traduit cette centralité métier : la création d'une conversation est le seul cas du projet gardé par une règle métier non triviale (le middleware `requireSimpleFollow` impose qu'un membre suive l'autre avant toute prise de contact), tandis que la table `Conversation` est l'un des hubs du modèle relationnel, reliée à `User`, `Message`, et `Rating` via la règle de clôture.

Cette section présente les réalisations techniques selon la décomposition imposée par le référentiel : captures d'écran d'interfaces utilisateur, composants métier, composants d'accès aux données, et autres composants significatifs (handlers Socket.IO, middleware applicatif).

Le périmètre couvert dans la suite représente, en chiffres :

Élément	Volume
Schémas Prisma	3 modèles
Routes REST conv./messages	9 routes
Events Socket.IO bidirectionnels	10 events (4 entrants, 6 sortants)
Schémas de validation Zod	7 schémas (<code>conversation.validation.ts</code>)
Middlewares métier dédiés	1 fichier, 3 helpers
Composants React (<code>ConversationPage/</code>)	9 composants UI + 2 hooks locaux
Hooks React composés	8 hooks (1 façade <code>useMessaging</code> + 7 spécialisés)
Fichiers de tests dédiés	3 spec files (Node Test Runner)

Tous les extraits de code présentés ci-dessous proviennent du dépôt de production¹¹ et sont rendus dans un thème clair pour faciliter leur lecture par le jury.

¹⁰ Analyse rédigée après la période projet, dans le dépôt de documentation ; elle ne fait pas partie du livrable d'équipe.

¹¹Repo : github.com/O-clock-Dublin/projet-skillswap. Ce dépôt est figé pour la soutenance ; les éventuelles dettes techniques mentionnées sont assumées comme telles et reportées en V2.

8.1. Captures d'écran d'interfaces utilisateur

Les captures qui suivent ont été réalisées en environnement de développement local, dont le seed reproduit fidèlement les états observables en production¹². Elles couvrent les principales vues du parcours messagerie côté utilisateur connecté.

8.1.1. Liste des conversations — vue desktop

Capture à insérer : ../assets/captures-ui/07-conversation-list-desktop.png

Fig. 9. – Liste des conversations de l'utilisateur connecté. Composant racine : `ConversationSection.tsx`. Chaque entrée est rendue par `ConversationItem.tsx` (molécule), avec avatar, nom du correspondant, dernier message et horodatage relatif. L'interface présente les conversations triées par date du dernier message descendant. La sélection d'une entrée déclenche l'émission d'un event Socket.IO `conversation:join` qui inscrit le client dans la room dédiée à la conversation. L'état « vu / non vu » et le badge de message non lu sont gérés côté client via le hook `useConversationList`.

¹²Le seed dev (`backend/src/models/seeding.dev.ts`) initialise 41 utilisateurs, des relations de suivi, des conversations et des messages représentatifs. Compte de démonstration : `alice.dupont@example.com` / `password123`. Les avatars et données affichés sont fictifs.

8.1.2. Thread de message ouvert — vue desktop

Capture à insérer : ../assets/captures-ui/07-message-thread-desktop.png

Fig. 10. – Thread d’une conversation ouverte. Composants imbriqués : `MessageThread/index.tsx` encapsule `ThreadHeader`, `MessageList` (avec composant atomique `MessageBubble` pour chaque message) et `MessageInput`. La pagination cursor-based charge les messages plus anciens au scroll vers le haut.

La liste de messages est virtualisée en pagination ascendante (cursor-based) afin de supporter des conversations longues sans charger l’intégralité de l’historique. La saisie utilisateur transite par `MessageInput.tsx`, qui émet l’événement `Socket.IO message:send` plutôt qu’une requête REST — choix architectural justifié dans la sous-section 8.4.

8.1.3. Dialogue d’évaluation après clôture

Capture à insérer : ../assets/captures-ui/07-rating-dialog.png

Fig. 11. – Dialogue d’évaluation déclenché à la clôture d’une conversation par l’autre participant (`RatingDialog.tsx`). Note de 1 à 5 et commentaire facultatif. Bloqué côté serveur si l’utilisateur a déjà évalué le membre cible (contrainte d’unicité Prisma `@@unique([evaluatorId, evaluatedId])`).

L'évaluation est fonctionnellement couplée à la clôture : l'événement `conversation:closed` déclenche l'apparition du dialogue, ce qui garantit qu'un membre ne peut évaluer qu'après une interaction effective. L'adaptation mobile (responsive Tailwind, navigation stack-based au lieu de split-view sur écran étroit) et le dialogue de création (`NewConversationDialog.tsx`, avec gating front+back `requireSimpleFollow`) sont visibles directement sur skill-swap.fr et présentés en démo orale.

8.2. Composants métier — orchestrateur `useMessaging` et optimistic UI

L'orchestration de la messagerie côté frontend repose sur la **composition de hooks React natifs** plutôt que sur une bibliothèque tierce de gestion d'état serveur. TanStack Query avait été envisagé (ADR-004¹³) mais n'a finalement pas été intégré : la composition de hooks natifs a été retenue à l'usage, pour trois raisons — la maîtrise fine du cycle de vie des sockets et des effets, la réduction de la surface de dépendances externes, et l'apprentissage approfondi des hooks React natifs (`useState`, `useEffect`, `useCallback`, `useRef`, `AbortController`).

8.2.1. Architecture des hooks — le pattern de composition

Le hook racine `useMessaging.ts` (139 LOC) compose sept hooks spécialisés, chacun responsable d'une dimension fonctionnelle isolée :

Hook	Responsabilité	LOC approx.
<code>useConversationList</code>	Charge la liste, expose les mutateurs (<code>addConversation</code> , <code>updateConversationLastMessage</code> , etc.)	95
<code>useSelectedConversation</code>	Mémoire de l'identifiant sélectionné, dérive l'objet conversation depuis la liste	64
<code>useConversationMessages</code>	Pagination cursor-based, optimistic UI (ajout temporaire avant DTO serveur)	163
<code>useFollowedUsers</code>	Liste des utilisateurs suivis (pour la création de nouvelle conversation)	30
<code>useGlobalSocket</code>	Listeners globaux : <code>conversation:updated</code> , <code>conversation:closed</code> , <code>conversation:new</code>	94
<code>useConversationActions</code>	Handlers UI ; instancie <code>useSocket(selectedConvId)</code> pour <code>message:send</code> et <code>conversation:close</code>	184
<code>useMessagingScroll</code>	Synchronisation du scroll auto sur réception de message + scroll inverse au load-more	111

¹³ `docs/documentation-implementation/arc42/09-decisions/004-tanstack-query.md` — dépôt d'équipe, branche `Documentation`.

Chaque hook est testable indépendamment et maintient un périmètre de responsabilités strict. Cette discipline permet d'isoler les régressions lors d'une évolution future et de raisonner localement sur chaque mécanisme — typiquement, un bug d'optimistic UI se cherche dans `useConversationMessages` sans devoir comprendre la pagination ou la navigation.

8.2.2. Hook racine — composition

```
// frontend/src/hooks/useMessaging.ts (extrait simplifié — 139 LOC au total)
export function useMessaging() {
  const { user } = useAuth();

  const {
    conversations,
    addConversation,
    updateConversationLastMessage,
    updateConversationStatus,
    removeConversation,
  } = useConversationList();

  const {
    selectedConvId,
    setSelectedConvId,
    selectedConv,
    clearSelection,
  } = useSelectedConversation(conversations);

  const {
    messages,
    hasMore,
    loadMore,
    addOptimisticMessage,
  } = useConversationMessages({
    conversationId: selectedConvId,
    limit: 30,
  });

  const { followedUsers, fetchFollowedUsers } = useFollowedUsers();

  const {
    onConversationUpdate,
    onConversationClosed,
    onConversationNew,
  } = useGlobalSocket();

  // Réagit à un nouveau message reçu sur n'importe quelle conversation
  useEffect(() => {
    onConversationUpdate((conversationId, lastMessage) => {
      updateConversationLastMessage(conversationId, lastMessage);
    });
  }, [onConversationUpdate, updateConversationLastMessage]);

  // Réagit à la clôture d'une conversation par l'autre participant
  useEffect(() => {
```

```

onConversationClosed((conversationId, closedBy) => {
  updateConversationStatus(conversationId, "Close");
  if (closedBy && closedBy.id !== user?.id) {
    toast.info(`${closedBy.firstname} à clôturer un échange`);
  }
  if (conversationId === selectedConvId) {
    clearSelection();
  }
});
}, [onConversationClosed, updateConversationStatus,
  selectedConvId, clearSelection, user]);

// Réagit à une nouvelle conversation reçue (premier message d'un autre membre)
useEffect(() => {
  onConversationNew((conversation) => {
    addConversation(conversation);
    toast.info(
      `${conversation.participant?.firstname} a démarré un nouvel échange`,
    );
  });
}, [onConversationNew, addConversation]);

// Composition complète des actions (signature réelle ; props omises ici)
const actions = useConversationActions({
  selectedConvId,
  selectedConv: selectedConvWithMessages,
  setSelectedConvId,
  addConversation,
  updateConversation,
  removeConversation,
  clearSelection,
  fetchFollowedUsers,
  addMessage,
  addOptimisticMessage,
});

return {
  conversations,
  selectedConv,
  messages,
  hasMore,
  loadMore,
  followedUsers,
  fetchFollowedUsers,
  ...actions,
};
}

```

L'effet de bord crucial est l'enregistrement des trois listeners globaux (`onConversationUpdate`, `onConversationClosed`, `onConversationNew`) qui maintiennent la liste de conversations synchronisée même quand l'utilisateur n'est pas en train de regarder la conversation concernée. Cette logique exploite le modèle de rooms Socket.IO : chaque utilisateur connecté rejoint automatiquement sa room personnelle

`user:${userId}` (cf. sous-section 8.4), ce qui permet au serveur de pousser les notifications de mise à jour ciblées.

8.2.3. Optimistic UI — gestion du `tempId` négatif

```
// frontend/src/hooks/messaging/useConversationActions.ts (extrait, l.100-118)
const handleSendMessage = useCallback(
  async (content: string) => {
    if (!selectedConvId || !content.trim()) return;

    // tempId négatif : marqueur d'un message en attente de réponse serveur
    const tempId = -Date.now();
    const sender = user
      ? {
          id: user.id,
          firstname: user.firstname,
          lastname: user.lastname,
          avatarUrl: user.avatarUrl,
        }
      : undefined;

    addOptimisticMessage(tempId, content, sender);
    socketSendMessage(content); // wrapper de socket.emit("message:send", ...)
  },
  [selectedConvId, socketSendMessage, addOptimisticMessage, user],
);
```

Le pattern `optimistic UI` consiste à insérer immédiatement le message dans la liste locale avec un identifiant négatif (`-Date.now()`), puis à laisser l'événement Socket.IO `message:new` émis par le serveur ajouter le DTO définitif (avec identifiant entier positif assigné par PostgreSQL). La déduplication exploite un filtre côté listener `onMessage` du hook `useConversationActions` : si `newMessage.sender?.id === user?.id`, le message est **ignoré** lors du retour serveur — l'optimistic local reste en place et sert d'affichage final.

```
// useConversationActions.ts:57-65 - déduplication des messages propres
useEffect(() => {
  onMessage((newMessage) => {
    if (newMessage.sender?.id === user?.id) {
      return; // déjà ajouté en optimistic, on ignore le retour serveur
    }
    addMessage(newMessage);
  });
}, [onMessage, addMessage, user?.id]);
```

L'avantage utilisateur est immédiat : la latence perçue de l'envoi est quasi-nulle, ce qui correspond à l'expérience attendue d'une messagerie moderne. Le compromis assumé : en cas d'échec réseau silencieux, le message optimistic reste affiché avec son `tempId` négatif (pas de réconciliation automatique vers un état d'erreur dans le code actuel). Une amélioration V2 consiste à introduire un timeout de réconciliation et un état visuel « en cours d'envoi / échec » — point identifié dans la dette technique en fin de section.

8.3. Composants d'accès aux données — services Prisma

L'accès à la base de données passe par Prisma comme ORM type-safe (cf. ADR-003¹⁴). Les services backend exposent les opérations métier en masquant la mécanique des requêtes, ce qui permet aux contrôleurs et handlers Socket.IO de se concentrer sur la logique de validation et la diffusion des events.

8.3.1. Création atomique d'un message + mise à jour de la conversation

L'envoi d'un message implique deux écritures distinctes : insertion en table `Message` et mise à jour du champ `updatedAt` de la `Conversation` associée (pour la conserver en tête de liste). Plutôt qu'une transaction, le code utilise un `Promise.all` qui parallélise les deux requêtes — acceptable car elles ne sont pas mutuellement dépendantes, et la durée de fenêtre d'incohérence est de l'ordre de la milliseconde.

```
// backend/src/realtime/socket.ts (extrait du handler message:send)
const [msg] = await Promise.all([
  prisma.message.create({
    data: {
      conversationId,
      senderId: userId,
      receiverId,
      content,
    },
    select: {
      id: true,
      content: true,
      createdAt: true,
      sender: {
        select: {
          id: true,
          firstname: true,
          lastname: true,
          avatarUrl: true,
        },
      },
    },
  }),
  prisma.conversation.update({
    where: { id: conversationId },
    data: { updatedAt: new Date() },
  }),
]);
```

Le `select` explicite est un pattern récurrent dans le code prod : il limite les champs retournés au strict nécessaire pour le DTO réseau, ce qui évite à la fois le sur-fetch (gain de performance) et la fuite involontaire de champs sensibles (par exemple `password`, `email`, ou les champs liés au système de tokens). Cette discipline transversale relève autant de la performance que de la sécurité.

¹⁴ docs/documentation-implementation/arc42/09-decisions/003-prisma.md — dépôt d'équipe, branche `Documentation`.

8.3.2. Pagination cursor-based des messages

```
// backend/src/services/message.service.ts (extrait –
// getConversationMessagesService)
const limit = Math.min(Math.max(params.limit ?? 50, 1), 100);

const messages = await prisma.message.findMany({
  where: {
    conversationId,
    // Cursor : récupère uniquement les messages plus anciens que le cursor
    ...(params.cursor ? { id: { lt: params.cursor } } : {}),
  },
  orderBy: { id: 'desc' },
  take: limit,
  select: {
    id: true,
    content: true,
    createdAt: true,
    sender: {
      select: { id: true, firstname: true, lastname: true, avatarUrl: true },
    },
  },
});

// Reverse pour ordre chronologique côté front (oldest → newest)
const data = messages.slice().reverse().map(/* ... mapping DTO ... */);

// nextCursor null = plus de messages à charger
const nextCursor =
  messages.length > 0 ? messages[messages.length - 1].id : null;
```

Le choix d'une pagination **cursor-based** (par identifiant de message, clause Prisma `id: { lt: cursor }`) plutôt qu'**offset-based** (par numéro de page) est motivé par deux considérations : la stabilité face à l'insertion de nouveaux messages pendant la navigation (un offset sauterait des messages, un cursor reste stable), et la performance sur des tables longues (l'offset force PostgreSQL à scanner les lignes ignorées, le cursor permet un index seek direct via la clé primaire). Pour une messagerie où les messages s'accumulent en continu, le cursor est l'approche correcte. La clause `limit` est bornée à [1, 100] côté serveur (`Math.min(Math.max(limit ?? 50, 1), 100)`) pour empêcher tout abus côté client.

8.3.3. Schéma Prisma — extrait des modèles principaux

Le schéma Prisma intégral des trois modèles directement liés à la messagerie (`Conversation`, `UserHasConversation`, `Message`) plus `Follow` (gating) est reproduit en **annexe A**. Les choix de modélisation traduisent plusieurs décisions architecturales : la table de jonction `UserHasConversation` permet une relation N-N entre utilisateurs et conversations (donc support natif de conversations de groupe en V2 sans changement de schéma), la cascade `onDelete: Cascade` garantit l'absence de messages orphelins en cas de suppression d'utilisateur, le statut `StatusOfConversation` (`Open` / `Close`) matérialise le cycle de vie de l'échange jusqu'à sa clôture (qui déclenche le dialogue d'évaluation), et la pagination cursor-based exploite la clé primaire `id` (autoincrement)

comme curseur naturel — sans nécessiter d'index composite supplémentaire. La limite de longueur de message (2000 caractères) est appliquée côté serveur dans le handler Socket.IO et dans le schéma Zod `CreateMessageSchema`, sans contrainte SQL explicite — trade-off de simplicité assumé.

8.4. Autres composants — Socket.IO server et middleware métier

Cette dernière sous-section regroupe les composants qui ne relèvent ni strictement de l'UI, ni des services d'accès aux données : le serveur Socket.IO et les middlewares applicatifs. Ils incarnent l'épine dorsale temps réel de la messagerie et la garde métier qui en conditionne l'accès.

8.4.1. Authentification Socket.IO par cookie JWT

L'authentification du client Socket.IO réutilise le même cookie HTTP-only `accessToken` que les routes REST, plutôt qu'un mécanisme distinct (token en query string par exemple). Ce choix élimine une surface d'attaque (le token n'est jamais exposé dans une URL ou loggué par un proxy) et simplifie la session côté frontend (un seul flux de rafraîchissement à gérer).

```
// backend/src/realtime/socket.ts (extrait, 1.88-122)
io.use((socket, next) => {
  try {
    const cookieHeader = socket.handshake.headers.cookie ?? "";
    const cookies = cookie.parse(cookieHeader);
    const token = cookies.accessToken;
    if (!token) return next(new Error("Unauthorized"));

    const decoded = jwt.verify(token, config.jwtSecret) as jwt.JwtPayload;
    const userId = Number(decoded.id ?? decoded.userId ?? decoded.sub);

    if (!Number.isInteger(userId) || userId <= 0) {
      return next(new Error("Unauthorized"));
    }

    socket.data.userId = userId;
    next();
  } catch {
    next(new Error("Unauthorized"));
  }
});

io.on("connection", (socket) => {
  // Inscription automatique à la room personnelle
  socket.join(`user:${socket.data.userId}`);

  socket.on("conversation:join", async (payload) => {
    /* validation + inscription room conversation */
  });

  socket.on("message:send", async (payload) => {
    /* validation + persistance + diffusion */
  });
});
```

```

socket.on("conversation:close", async (payload) => {
  /* validation + update status + diffusion */
});

socket.on("conversation:leave", (payload) => {
  /* leave room conversation (silencieux) */
});
});

```

À la connexion, chaque socket rejoint automatiquement sa room personnelle `user:${userId}`. Cette room est utilisée par le serveur pour pousser des notifications (nouvelle conversation reçue, mise à jour de la liste, fermeture par l'autre participant) qui doivent atteindre l'utilisateur indépendamment de la conversation qu'il est en train de visualiser.

8.4.2. Middleware métier `requireSimpleFollow` — gating de la création de conversation

```

// backend/src/middlewares/conv.middleware.ts:78-115
export const requireSimpleFollow = async (
  req: Request,
  _res: Response,
  next: NextFunction,
) => {
  try {
    const receiverId = Number(req.body.receiverId) || Number(req.paramsId);
    const senderId = req.userId;

    if (!Number.isInteger(receiverId) || receiverId <= 0) {
      return next(
        new BadRequestError('receiverId is required and must be a valid id.'),
      );
    }

    if (receiverId === senderId) {
      return next(
        new BadRequestError('You cannot start a conversation with yourself.'),
      );
    }

    const follow = await prisma.follow.findFirst({
      where: { followerId: senderId, followedId: receiverId },
      select: { followerId: true, followedId: true },
    });

    if (!follow) {
      return next(new ForbiddenError('You can only message users you follow.'));
    }

    next();
  } catch (e) {
    next(e);
  }
}

```



```
}
};
```

Ce middleware applique trois contrôles successifs : la validité du `receiverId` (entier positif), l'absence d'auto-conversation (impossible de se contacter soi-même), et la présence d'une relation de suivi `Follow` du `sender` vers le `receiver`. Cette dernière règle est **la plus importante du produit** : elle garantit qu'aucun spam direct n'est possible entre membres qui ne se sont pas mutuellement signalé un intérêt par le suivi. Le contournement d'un front compromis est donc bloqué au niveau backend.

8.4.3. Modèle de rooms Socket.IO

Le serveur exploite deux types de rooms simultanément, chacune avec un rôle bien défini :

Room	Membres	Usage
<code>user:\${userId}</code>	Toutes les sockets du même utilisateur (multi-onglets)	Notifications globales : <code>conversation:updated</code> <code>conversation:closed</code> <code>conversation:new</code>
<code>conversation:\${id}</code>	Participants ayant émis <code>conversation:join</code>	Diffusion ciblée : <code>message:new</code> <code>conversation:joined</code> <code>conversation:closed</code>

Le cloisonnement par rooms réalise un contrôle d'accès au niveau du transport : un message émis dans la room `conversation:42` n'est diffusé qu'aux participants ayant explicitement rejoint cette room, à condition qu'ils soient déjà passés par la validation participant du handler `conversation:join`. Un membre tiers ne peut pas écouter passivement les messages d'autrui.

8.4.4. Diagramme de séquence — envoi d'un message

Le flux complet d'un envoi de message, depuis la saisie utilisateur jusqu'à la confirmation chez les deux participants, est décrit dans le diagramme de séquence ci-après. Il a été produit avec PlantUML pendant le projet et fait partie du livrable d'équipe mergé sur `main`¹⁵.

¹⁵Source et rendu : `docs/uml/sequence/conversation.puml` et `conversation.png`, dépôt d'équipe.

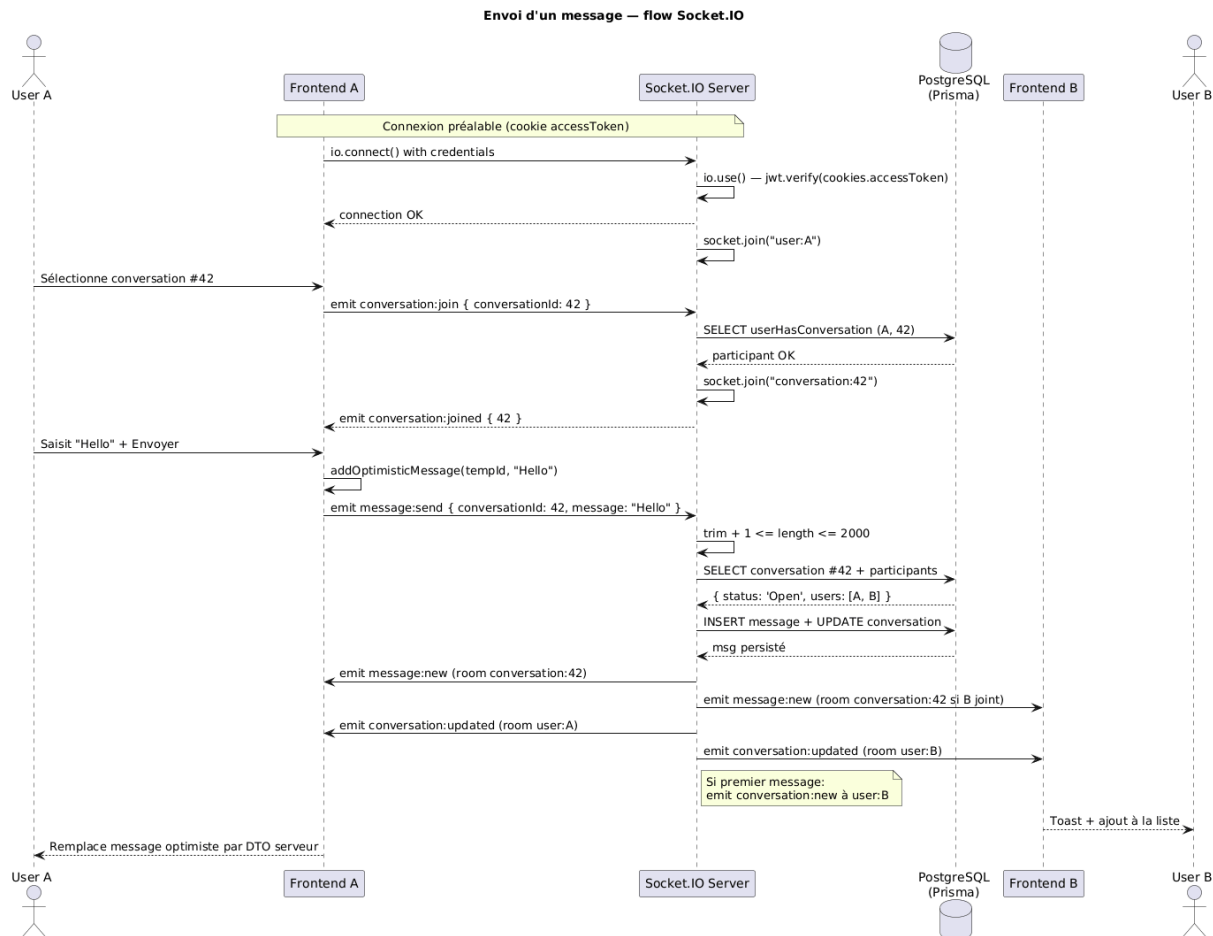


Fig. 12. – Diagramme de séquence de l'envoi d'un message — flux complet : authentification socket, optimistic UI, validation serveur, persistance Prisma, diffusion multi-rooms.

8.5. Bilan technique de la section

Cette section a couvert en profondeur la fonctionnalité messagerie selon les quatre axes imposés par le référentiel — interfaces utilisateur, composants métier, accès aux données, autres composants. Les compétences professionnelles mobilisées dans la réalisation se cartographient ainsi :

Compétence CDA	Mise en œuvre
CP2 — Développer des interfaces utilisateur	9 composants React (<code>ConversationPage/</code>) avec composition Atomic Design (atoms via <code>shadcn/ui</code> , molécules <code>ConversationItem</code> et <code>MessageBubble</code> , organisme <code>MessageThread</code> sous-composé en 5 fragments), responsive Tailwind, accessibilité ARIA.
CP3 — Développer des composants métier	Composition de 8 hooks React (orchestrateur + 7 spécialisés), pattern optimistic UI avec <code>tempId</code> négatif, gestion fine du cycle de vie des sockets via <code>useSocket(conversationId)</code> .

CP6 — Définir l'architecture logicielle	Architecture hybride REST + WebSocket assumée et documentée (ADR-011, formalisé a posteriori pour ce dossier), séparation claire des responsabilités entre handlers Socket et services métier, modèle de rooms à deux niveaux (room utilisateur + room conversation).
CP7 — appliqué à la messagerie	3 modèles dédiés (<code>Conversation</code> , <code>UserHasConversation</code> , <code>Message</code>), cascade de suppression sécurisée, pagination cursor-based exploitant la PK <code>id</code> , longueur de message bornée côté serveur (2000 caractères) et côté Zod.
CP8 — Développer des composants d'accès aux données	Services Prisma type-safe avec <code>select</code> explicite anti-overfetch, pagination cursor-based, parallélisation <code>Promise.all</code> sur écritures indépendantes.

8.5.1. Dette technique assumée

Les zones de fragilité connues sont assumées et reportées en V2 :

- **Handler** `message:send` **dense** (`socket.ts:167-347`) — fetch, vérifs, persistance et 3 émissions concentrés dans une seule fonction ; refactorisation vers `services/message.service.ts` prévue en V2.
- **Validation Socket non-Zod** — bornes manuelles (`Number.isInteger`, longueur) au lieu de schémas Zod ; trade-off latence assumé, harmonisation possible en V2.
- **Aucun test automatisé côté frontend** — le livrable ne contient ni test unitaire, ni test E2E : le scénario `alice→bob` à deux navigateurs n'est validé que manuellement (cf. section 11). L'outillage correspondant (Vitest, Playwright) est décidé dans l'ADR-010 mais n'a pas été intégré avant la soutenance.
- **Couverture non mesurée** — aucun outil de couverture n'est branché, ni en local ni en CI ; les sept fichiers de tests backend ne sont donc pas quantifiés.

Ces dettes n'ont pas affecté la stabilité observée en production sur `skill-swap.fr` depuis la mise en service.

9. Éléments de sécurité

Les choix de sécurité transversale (hashing argon2, JWT en cookies `httpOnly`, CORS, HTTPS) ont été présentés en sous-section 7.4. La présente section porte sur les contrôles **spécifiques à la messagerie** — la fonctionnalité retenue comme la plus représentative — qui combinent authentification du canal WebSocket, autorisation contextuelle à chaque event, validation des entrées et cloisonnement par rooms. Ces contrôles forment une chaîne où chaque maillon est nécessaire pour qu'un message émis par un membre parvienne effectivement, et seulement, à son destinataire légitime.

9.1. Authentification du canal Socket.IO

Le serveur Socket.IO réutilise le cookie `accessToken` posé par l'authentification REST plutôt qu'un mécanisme distinct (token en query string ou auth manuelle). Au handshake, un middleware de session (`io.use`) extrait le cookie, vérifie la signature JWT et stocke l'identifiant utilisateur dans `socket.data.userId`. En cas d'échec — cookie absent, signature invalide, identifiant non entier positif — la connexion est rejetée avec `next(new Error('Unauthorized'))` avant qu'aucun event applicatif ne puisse être émis ou reçu.

Ce choix élimine deux surfaces d'attaque par rapport à un token en URL : le token n'est jamais exposé dans une chaîne de connexion ni loggué par un proxy intermédiaire, et la session reste cohérente entre les flux REST et WebSocket (un seul flux de rafraîchissement à gérer côté frontend). Le code complet de ce handshake est reproduit en **annexe B**.

9.2. Contrôles d'accès appliqués à chaque event

Contrôle	Effet	Réf. code
Auth handshake	Refus de connexion si JWT manquant ou invalide.	<code>socket.ts:88-122</code>
Vérif participant <code>conversation:join</code>	Refus de rejoindre une room conversation si l'utilisateur n'est pas dans <code>UserHasConversation</code> pour cette conversation.	<code>socket.ts:136-152</code>
Validation bornes message	Refus si <code>content</code> absent, vide après <code>trim()</code> , ou supérieur à 2000 caractères.	<code>socket.ts:167-186</code>
Refus si conversation <code>Close</code>	Aucun message accepté sur une conversation clôturée — le statut est lu en base avant persistance.	<code>socket.ts:214-220</code>
Vérif participant <code>message:send</code>	Refus si l'émetteur n'est pas participant de la conversation cible (recheck explicite, indépendant de <code>conversation:join</code>).	<code>socket.ts:222-230</code>

Cloisonnement par rooms	Diffusion ciblée — un message émis dans <code>conversation:42</code> n'atteint que les sockets ayant rejoint cette room précise. Inscription à la room personnelle à la connexion ; inscription à la room conversation seulement après vérification participant.	<code>socket.ts:128, 155</code>
-------------------------	--	---------------------------------

À ces six contrôles côté Socket s'ajoute **en amont** le gating métier appliqué au moment de la création de la conversation par REST : le middleware `requireSimpleFollow` (`conv.middleware.ts:78-115`) exige que l'émetteur suive le destinataire avant que la conversation puisse exister. Sans relation de suivi, aucune conversation n'est créée, donc aucun message ne peut transiter — le contrôle Socket de l'étape 5 ne sera jamais sollicité car il n'y aura pas de couple conversation/participant à vérifier. Cette antériorité du gating REST sur le pipeline Socket constitue le verrou métier le plus important du produit.

9.3. Défense en profondeur — front, back, base

Trois barrières indépendantes appliquent les mêmes règles à des niveaux distincts du système, ce qui garantit que la compromission d'une couche ne suffit pas à contourner les contrôles.

Niveau	Mesures
Frontend	Le dialogue de création de conversation ¹⁶ ne propose que les utilisateurs suivis (filtrage par <code>useFollowedUsers</code>). <code>MessageInput.tsx</code> désactive le champ de saisie quand <code>conversationStatus === 'Close'</code> . Ces mesures relèvent de l'UX — elles guident le membre légitime mais ne sont pas des contrôles de sécurité réels (un client compromis peut les contourner).
Backend	Les six contrôles Socket listés ci-dessus + le gating REST <code>requireSimpleFollow</code> forment la vraie barrière de sécurité . Toute requête mal formée, tout payload hors-bornes, tout participant non légitime est refusé avec une erreur explicite avant tout effet de bord.
Base de données	Trois contraintes d'unicité matérialisent au niveau base les règles non contournables : <code>@@unique([followedId, followerId])</code> sur <code>Follow</code> , <code>@@unique([evaluatorId, evaluatedId])</code> sur la table <code>evaluation</code> , clé primaire composite <code>@@id([userId, conversationId])</code> sur <code>UserHasConversation</code> . Ces contraintes prennent le relais en cas de bug applicatif (idempotence garantie sur les insertions concurrentes).

¹⁶ `frontend/src/components/organisms/ConversationPage/NewConversationDialog.tsx`

9.4. Dettes de sécurité assumées

Quatre zones de fragilité ont été identifiées en audit interne et sont documentées plutôt que masquées. Aucune ne remet en cause la stabilité fonctionnelle observée en production, mais toutes constituent des chantiers de durcissement à programmer en V2.

Dettes	Impact et plan V2
Helmet installé non monté	La dépendance <code>helmet@8.1.0</code> est listée dans <code>backend/package.json</code> mais jamais appliquée dans <code>backend/src/app.ts</code> . Mitigation actuelle : Nginx pose en façade <code>X-Frame-Options</code> , <code>X-Content-Type-Options</code> , <code>X-XSS-Protection</code> et HSTS. V2 : ajouter <code>app.use(helmet())</code> dans Express (correction d'une ligne).
Rate-limiting absent	Aucune protection applicative contre l'abus en volume — ni <code>express-rate-limit</code> , ni <code>limit_req</code> Nginx. Risque : brute-force sur l'authentification, abus d'envoi de messages. V2 : <code>express-rate-limit</code> sur les routes <code>/api/v1/auth/{login,register,refresh}</code> (5 tentatives / 15 min par IP).
Content Security Policy absente	Aucune directive <code>Content-Security-Policy</code> posée, ni côté Express ni côté Nginx. Le frontend Next.js n'est donc pas protégé contre l'injection de scripts tiers. V2 : CSP stricte côté Nginx en mode <code>Report-Only</code> d'abord, puis enforcement après recensement des inline-scripts légitimes.
Validation Socket sans Zod	Les events Socket utilisent une validation manuelle (<code>Number.isInteger</code> , bornes de longueur) plutôt que les schémas Zod employés sur les routes REST. Trade-off latence assumé pour le handler <code>message:send</code> . V2 : harmonisation Zod si la latence mesurée le permet.

10. Plan de tests

10.1. Stratégie de tests

10.2. Tests backend (Node Test Runner natif)

Sept suites d'intégration ont été écrites pour le Node Test Runner natif, sans framework tiers, couvrant les contrôleurs (authentification, profil, suivi, recherche, conversations, messages) et les events Socket.IO.

Cinq de ces sept suites échouent. La cause est identifiée et unique : un bug dans la fixture globale, qui ne garantit pas la création de la donnée de rôle avant les utilisateurs de test — les utilisateurs sont alors insérés avec une clé étrangère invalide et toute la suite tombe. Le défaut est dans le code de test, pas dans le code applicatif, et il est commun aux cinq suites concernées.

Cette dette est assumée : elle n'a pas été corrigée avant la soutenance, la priorité ayant été donnée à la finition fonctionnelle. Elle est documentée comme premier chantier de reprise, et sa correction est de faible ampleur — réordonner le `beforeAll` global. Conséquence honnête à en tirer : le projet dispose d'une **intention** de couverture backend, pas d'un filet de sécurité opérationnel, et aucune couverture n'a jamais été mesurée.

10.3. Absence de tests automatisés côté frontend

10.4. Plan de tests fonctionnels

Type	Utilisateur	Fonctionnalité	Page	Objectif	Version	Détails
Manuel	Membre	Inscription	/inscription	Créer un compte avec email valide	v1.0	Email + mot de passe ≥ 8 + confirmation
Manuel	Membre	Connexion	/connexion	Se connecter avec identifiants valides	v1.0	Cookie httpOnly accessToken + refreshToken posés
Manuel	Membre	Recherche	/recherche	Trouver un	v1.0	Debounce 300ms

				membre par compétence	+ filtre catégorie
Manuel	Membre	Follow	/profil/:id	Suivre un autre membre	v1.0 Pré- requis pour messagerie
Manuel	Membre	Conversation	/conversation	Démarrer une conversation	v1.0 Refus si pas de follow
Manuel	Membre	Message temps réel	/conversation	Recevoir un message en live	v1.0 Socket.IO event message:new
Manuel	Membre	Évaluation	/conversation	Noter un membre après échange	v1.0 Pré- requis follow + dialog au close

10.5. Manques honnêtes (à mentionner)

11. Jeu d'essai

11.1. Scénario testé

11.2. Étape 1 — Alice suit Bob (POST /follows/:bobId/follow)

Données d'entrée	Données attendues	Données obtenues	Analyse
POST /api/v1/follows/42/follow Cookie : accessToken (Alice) Body : aucun	HTTP 201 Created <pre>{ success: true, data: { ... } }</pre> Ligne créée dans table <code>follow</code>		

11.3. Étape 2 — Alice crée la conversation (POST /api/v1/conversations)

Données d'entrée	Données attendues	Données obtenues	Analyse
POST /api/v1/conversations Cookie : accessToken (Alice) Body : <pre>{ "title": "Cours guitare", "receiverId": 42 }</pre>	HTTP 201 Created <pre>{ success: true, data: { id, title, status: 'Open', users: [...] } }</pre> Middleware <code>requireSimpleFollow</code> passe (follow existe à l'étape 1) 2 lignes créées dans <code>user_has_conversation</code>		

11.4. Étape 3 — Alice envoie le premier message via Socket.IO

Données d'entrée	Données attendues	Données obtenues	Analyse
Event : <code>message:send</code> Payload :	Côté DB : 1 ligne créée dans <code>message</code> , conversation		

<pre>{ conversationId: 17, message: "Salut Bob !" }</pre> Socket connecté avec cookie accessToken (Alice)	updatedAt modifié Côté Socket : 3 events émis • <code>message:new</code> sur room <code>conversation:17</code> • <code>conversation:updated</code> sur <code>user:1</code> et <code>user:42</code> • <code>conversation:new</code> sur <code>user:42</code> (premier message)		
--	---	--	--

11.5. Étape 4 — Bob reçoit la notification temps réel

Données d'entrée	Données attendues	Données obtenues	Analyse
Bob a la page / conversation ouverte, mais sur une AUTRE conversation Socket connecté avec cookie accessToken (Bob)	Apparition du toast « Alice a démarré un nouvel échange » Nouvelle conversation visible en haut de la liste <code>useGlobalSocket.onConversationNew</code> déclenché		

11.6. Étape 5 — Optimistic UI résolu côté Alice

Données d'entrée	Données attendues	Données obtenues	Analyse
Alice a vu son message s'afficher immédiatement (optimistic) Réception de l'événement <code>message:new</code> avec id réel	Le message optimistic (id temporaire) est remplacé par le message persisté (id base réel, timestamp serveur autoritatif)		

11.7. Captures d'écran annexées

11.8. Limites du jeu d'essai

12. Veille de sécurité

12.1. Périmètre et constat

Aucune veille de sécurité formalisée n'a été conduite pendant le projet : ni journal dédié, ni source suivie, ni périodicité définie, ni outil automatisé de suivi des vulnérabilités — le dépôt ne contient aucun fichier `dependabot.yml`, aucune configuration Renovate, aucun rapport d'audit de dépendances. La sécurité a été traitée par des choix d'implémentation et par une réaction ponctuelle à un incident, décrits ci-dessous. C'est une lacune identifiée, dont la remédiation est proposée en fin de section.

12.2. Une vulnérabilité identifiée et traitée pendant le projet

Le fichier d'environnement `devops/.env.docker` est ajouté au suivi Git le 12 janvier, puis complété d'un `JWT_SECRET` en clair le 13. Je l'ai retiré du suivi le 14 janvier (commit `6bd3e25`, « `fix(security): remove .env.docker from git tracking` »), et le `.gitignore` couvre désormais ce chemin, ce qui prévient la récurrence.

Analyse de la mesure. Retirer un fichier du suivi empêche les commits ultérieurs de le réintroduire, mais ne supprime pas la valeur de l'historique déjà écrit. Une remédiation complète aurait supposé une rotation du secret côté serveur, et le cas échéant une réécriture d'historique. Cette limite est assumée.

12.3. Mesures de sécurité retenues pendant le projet

- **argon2** pour le hachage des mots de passe (`auth.service.ts:21`) — fonction à coût mémoire, résistante aux attaques par matériel dédié.
- **JWT en cookies** `httpOnly` + `secure` + `sameSite=strict` — le jeton n'est pas accessible au JavaScript client, ce qui neutralise son vol par XSS. L'ADR-007 rejette explicitement `localStorage` pour ce motif.
- **Rotation du refresh token** à chaque renouvellement, avec invalidation du précédent (`auth.service.ts:103-108`).
- **Origine CORS unique**, fournie par variable d'environnement, appliquée à Express et au handshake Socket.IO ; en production, le serveur refuse de démarrer si elle est absente.
- **Validation systématique des entrées** par schémas Zod, et requêtes paramétrées via l'ORM contre l'injection SQL.

Ces choix n'ont pas fait l'objet d'une justification écrite pendant le projet — la documentation de sécurité d'époque mentionnait même bcrypt alors que le code utilise argon2. L'écart a été relevé et corrigé lors de la reprise documentaire pour ce dossier.

12.4. Analyse OWASP conduite sur l'application

La couverture du référentiel OWASP Top 10 est présentée en section 9. Quatre dettes y sont identifiées et assumées : Helmet installé mais non monté, absence de Content-Security-Policy, absence de limitation de débit, accessibilité non mesurée.

12.5. Ce que je mettrais en place

Outillage automatisé. Activer Dependabot ou Renovate sur le dépôt, avec regroupement des mises à jour mineures et de correctifs en une seule Pull Request périodique — pour éviter le bruit qui fait qu'on finit par ne plus les lire — et alerte immédiate, isolée, sur les avis critiques. La valeur de l'outil tient à ce réglage : mal configuré, il produit un flux que l'équipe apprend à ignorer.

Contrôle en intégration continue. Exécuter `npm audit` à chaque passage de pipeline, bloquant au-delà d'un seuil de sévérité défini — par exemple *high* sur les dépendances directes — avec une dérogation explicite et tracée pour les cas jugés non exploitables. La dérogation doit être écrite quelque part : un seuil qu'on contourne silencieusement ne protège plus.

Distinguer la vulnérabilité déclarée du risque réel. Une dépendance vulnérable n'est dangereuse que si le code appelle effectivement le chemin concerné. La démarche consiste à vérifier trois choses : si le paquet est une dépendance directe ou transitive ; s'il est chargé en production ou seulement en développement et en test ; et si la condition d'exploitation décrite par l'avis est réunie dans le contexte de l'application. Un paquet importé uniquement par un fichier de test n'expose pas le service ; à l'inverse, une bibliothèque traversée par chaque requête entrante mérite un traitement immédiat.

Sources suivies. Les bulletins de sécurité des projets critiques du socle — Next.js, Express, Prisma, Socket.IO —, la base GitHub Advisory, les avis de l'ANSSI et du CERT-FR, et les recherches en anglais sur les termes de vulnérabilité (`CVE`, `security advisory`, `GHSA`) associés aux paquets utilisés.

Périodicité. Revue hebdomadaire des alertes automatiques, revue mensuelle des versions majeures du socle. La revue mensuelle sert à décider des montées de version qui ne sont pas déclenchées par un avis mais par le risque de retard accumulé.

Traitement des dettes déjà identifiées. Monter Helmet côté Express, définir une Content-Security-Policy en mode observation avant enforcement, et ajouter une limitation de débit sur les routes d'authentification.

13. Difficultés rencontrées et solutions

Les difficultés relatées ici sont documentées par deux sources : le journal de bord tenu par l'équipe pendant le projet¹⁷ et l'historique du dépôt d'équipe.

13.1. La mise au point de l'environnement conteneurisé

La stabilisation de l'environnement Docker a occupé les premiers jours du sprint 1 et a demandé plusieurs itérations. Le journal de bord d'équipe enregistre le 14 janvier un « debug docker seeding + frontend qui restart en boucle » ; le même jour et le suivant, j'ai traité la configuration de production, les healthchecks, le support du rechargement à chaud (HMR) dans le conteneur, le passage des images de développement à Node 24, la correction du script d'attente de PostgreSQL — chemin absolu, ordonnancement du démarrage base/API — puis les volumes nommés et le montage sélectif de la configuration Nginx. Quinze commits sur trois jours ont été nécessaires pour obtenir un environnement stable.

Une régression est survenue une semaine plus tard : le 19 janvier, une erreur 502 bad gateway a été constatée en environnement conteneurisé ; elle provenait d'un port de proxy Nginx désaccordé avec le backend, que j'ai corrigé. L'infrastructure s'est révélée sensible à chaque ajout de service — Meilisearch le 22 janvier, la gestion des avatars le 29.

13.2. Le contrat d'interface entre le frontend et le backend

Le projet ne dispose pas de package de types partagé : les types sont déclarés des deux côtés. Cette duplication, assumée au cadrage, a fait du contrat d'interface la zone la plus instable du code — `api-client.ts` et `api-types.ts` totalisent soixante-trois retouches sur les quatre semaines. Chaque évolution du schéma Prisma se répercutait en plusieurs points du frontend, comme en témoignent les messages de commits (« align api-types with Prisma schema »).

Une difficulté connexe a été le rattrapage d'une convention de nommage non tranchée au cadrage : la coexistence du camelCase côté TypeScript et du snake_case côté base a nécessité, du 17 au 19 janvier, quatre commits correctifs et une migration dédiée (`fix_snake_case`), renommant notamment `avatarUrl` en `avatar_url`. Corriger une convention en cours de projet suppose d'intervenir simultanément en base, dans le backend et dans le frontend.

13.3. Une revue de code à mi-parcours

Le 20 janvier, j'ai conduit une revue systématique du frontend, organisée en quatre phases et documentée au fil de son avancement. Elle a identifié trente points de correction, dont des bugs de logique : mutation de tableau, problème de synchronisation asynchrone, mapping de données incomplet. Elle a également porté sur la cohérence des conventions — renommage des fichiers de composants en

¹⁷Tableur partagé, hors dépôt Git — cf. section 5.

PascalCase — et sur la factorisation, avec la mise en place d'un *factory pattern* pour les icônes. Cette journée entière consacrée à la qualité plutôt qu'à de nouvelles fonctionnalités a été un arbitrage assumé.

13.4. Le cycle de vie de la session

La gestion de session a été ajustée par retouches successives sur l'ensemble du projet : correction de la déconnexion et de l'inscription le 15 janvier, de l'expiration côté frontend le 20, enrichissement des réponses d'authentification et traitement des erreurs 401. Le fournisseur de contexte d'authentification compte vingt-quatre retouches.

La difficulté la plus tenace est apparue le dernier jour : la pose des cookies lors des redirections a nécessité quatre passes successives. La navigation côté client de Next.js ne déclenchant pas la prise en compte des cookies posés par le serveur, il a fallu forcer un rechargement complet via `window.location.href` pour la connexion, la déconnexion et la suppression de compte.

13.5. Les tests d'intégration backend

Cinq des sept suites de tests d'intégration échouaient à l'issue du projet. La cause a été identifiée et documentée dans la documentation qualité : les données de rôle ne sont pas créées avant les utilisateurs de test dans la fixture. Le diagnostic est établi — il s'agit d'un ordonnancement d'initialisation, non d'un défaut du code applicatif — mais le correctif n'a pas été appliqué avant la fin du projet.

13.6. Ce que ces difficultés ont en commun

Trois d'entre elles — le contrat d'interface, la convention de nommage, la gestion de session — ne relèvent pas d'un obstacle technique isolé mais d'une décision de cadrage insuffisamment tranchée au départ, dont le coût s'est étalé sur tout le projet. À l'inverse, la journée de revue de code du 20 janvier a montré qu'un temps consacré à la qualité en cours de route se rentabilise. J'en retiens qu'il vaut mieux fixer tôt les conventions transverses — nommage, types partagés, gestion de session — quitte à y consacrer une demi-journée de cadrage supplémentaire.

14. Conclusion

14.1. Bilan du projet

L'apothéose SkillSwap a livré, en quatre semaines, une plateforme web complète et déployée en production, intégrant les sept fonctionnalités du périmètre MVP cible : page d'accueil, authentification, profils avec compétences et disponibilités, moteur de recherche, suivi de profils, messagerie temps réel, système d'évaluation. Les deux fonctionnalités initialement classées comme « évolutions possibles » — messagerie temps réel via WebSocket et recherche avancée via Meilisearch — ont été promues au périmètre MVP en cours de projet et livrées en production. La couverture fonctionnelle atteinte dépasse donc le périmètre minimum acté en sprint 0.

L'équipe a appliqué la méthode Scrum sur quatre sprints d'une semaine, maintenu une qualité de code outillée (ESLint, Prettier, Husky), écrit sept suites de tests d'intégration backend, et conduit une mise en production sur VPS avec Docker, Nginx et certificat TLS renouvelé automatiquement.

14.2. Évolutions identifiées pour la V2

Quatre catégories de dettes techniques tracées au fil du projet :

Sécurité — monter Helmet (installé mais non monté), ajouter une limitation de débit sur les routes d'authentification, définir une Content Security Policy (en observation puis en application).

Tests — corriger le défaut de fixture qui met cinq des sept suites d'intégration en échec, puis mettre en place la couverture frontend décidée à l'ADR-010 (unitaire, E2E) et non implémentée.

DevOps — compléter le déploiement continu par une intégration continue (build, lint, tests), mettre en place supervision et sauvegarde automatique de la base.

Fonctionnel — blocage de profils, suggestion automatique de correspondances, groupes thématiques.

14.3. Apprentissage personnel

Ce projet a été l'occasion d'appliquer rigoureusement l'Atomic Design à un frontend de production — sept pages, dix-huit atomes, neuf molécules et trente-neuf fichiers d'organismes regroupés en cinq ensembles fonctionnels —, de concevoir et déployer une chaîne de production complète (images multi-étapes, reverse proxy, TLS), et d'amorcer une démarche de documentation continue en rédigeant une documentation Arc42 partagée à l'équipe — démarche que je poursuis dans mes projets suivants.

La reprise de ce dossier m'a par ailleurs conduit à vérifier systématiquement chaque affirmation documentaire contre le code livré. L'exercice a révélé des écarts, corrigés depuis. J'en retiens qu'une documentation n'a de valeur que si elle est vérifiable, et

qu'il vaut mieux assumer une dette identifiée que présenter une couverture qu'on n'a pas.

15. Lexique

Terme	Définition
argon2id	Algorithme de hashing de mots de passe gagnant du Password Hashing Competition 2015. Variante recommandée par l'OWASP.
Atomic Design	Méthodologie de design de composants UI proposée par Brad Frost, organisant les composants en atomes / molécules / organismes / templates / pages. Adoptée pour la structure du front SkillSwap.
Cookie httpOnly	Cookie HTTP non lisible par le JavaScript côté client. Protège contre le vol de session par XSS.
JWT (JSON Web Token)	Token d'authentification signé contenant les claims utilisateur. Utilisé comme accessToken dans SkillSwap.
Meilisearch	Moteur de recherche full-text open-source écrit en Rust. Utilisé pour l'indexation des membres SkillSwap.
Optimistic UI	Pattern UX consistant à afficher immédiatement le résultat d'une action côté client avant la confirmation serveur, pour fluidifier la perception.
Prisma	ORM TypeScript-first pour Node.js, génère un client typé depuis un schéma déclaratif. Utilisé pour PostgreSQL dans SkillSwap.
Refresh token rotation	Pattern d'authentification où chaque utilisation du refresh token le remplace par un nouveau, invalidant l'ancien. Limite la fenêtre de jeu.
Room (Socket.IO)	Mécanisme de groupement de sockets côté serveur, permettant de cibler l'émission d'événements à un sous-ensemble de clients.
Server Component (Next.js)	Composant React rendu côté serveur dans Next.js App Router, sans envoi de JavaScript au client.
Socket.IO	Bibliothèque JavaScript pour la communication temps réel bidirectionnelle entre client et serveur, basée sur WebSocket avec fallback HTTP long-poll.
Zod	Bibliothèque TypeScript de validation et inférence de schémas. Utilisée pour valider toutes les entrées REST côté backend SkillSwap.

16. Annexes

Les présentes annexes regroupent les éléments matériels demandés par le référentiel pour la **fonctionnalité la plus représentative** du projet — la messagerie temps réel. Elles permettent à l'évaluateur d'inspecter le code réel sans devoir alterner entre le dossier et le dépôt source.

16.1. Annexe A — Schéma Prisma de la fonctionnalité messagerie

Extrait de `backend/prisma/schema.prisma` couvrant les modèles directement impliqués dans la messagerie : `Conversation`, `UserHasConversation`, `Message`, `Follow` (gating), et les relations vers `User`.

```
model Conversation {
  id          Int          @id @default(autoincrement())
  status      StatusOfConversation @default(Open)
  title       String
  users       UserHasConversation[]
  messages    Message[]

  createdAt DateTime @default(now()) @map("created_at")
  updatedAt DateTime @default(now()) @updatedAt @map("updated_at")

  @@map("conversation")
}

enum StatusOfConversation {
  Open
  Close
}

model UserHasConversation {
  user      User          @relation(fields: [userId], references: [id],
onDelete: Cascade)
  userId    Int           @map("user_id")
  conversation Conversation @relation(fields: [conversationId], references:
[id], onDelete: Cascade)
  conversationId Int       @map("conversation_id")

  createdAt DateTime @default(now()) @map("created_at")
  updatedAt DateTime @default(now()) @updatedAt @map("updated_at")

  @@id([userId, conversationId])
  @@map("user_has_conversation")
}

model Message {
  id          Int          @id @default(autoincrement())
  sender      User          @relation("SenderUser", fields: [senderId],
references: [id], onDelete: Cascade)
  senderId    Int          @map("sender_id")
  receiver    User          @relation("ReceiverUser", fields: [receiverId],
```

```

references: [id], onDelete: Cascade)
  receiverId      Int          @map("receiver_id")
  content         String
  conversation     Conversation @relation(fields: [conversationId], references:
[id], onDelete: Cascade)
  conversationId  Int          @map("conversation_id")

  createdAt       DateTime      @default(now()) @map("created_at")
  updatedAt       DateTime      @default(now()) @updatedAt @map("updated_at")

  @@map("message")
}

model Follow {
  id          Int      @id @default(autoincrement())
  followed    User     @relation("FollowedUser", fields: [followedId], references:
[id], onDelete: Cascade)
  followedId  Int      @map("followed_id")
  follower    User     @relation("FollowerUser", fields: [followerId], references:
[id], onDelete: Cascade)
  followerId  Int      @map("follower_id")

  createdAt   DateTime @default(now()) @map("created_at")
  updatedAt   DateTime @default(now()) @updatedAt @map("updated_at")

  @@unique([followedId, followerId])
  @@map("follow")
}

```

16.1.1. Extraits SQL générés par Prisma — migrations clés

Migration `init_db` (création de la table `user`) — extrait du fichier

`backend/prisma/migrations/20260112133206_init_db/migration.sql` :

```

CREATE TABLE "user" (
  "id"          SERIAL NOT NULL,
  "firstname"   TEXT NOT NULL,
  "lastname"    TEXT NOT NULL,
  "email"       TEXT NOT NULL,
  "password"    TEXT NOT NULL,
  "address"     TEXT,
  "postal_code" INTEGER,
  "city"        TEXT,
  "age"         INTEGER,
  "avatarUrl"   TEXT,
  "description" TEXT,
  "role_id"     INTEGER NOT NULL,
  "created_at"  TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updated_at"  TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT "user_pkey" PRIMARY KEY ("id")
);

CREATE UNIQUE INDEX "user_email_key" ON "user"("email");

```

```
ALTER TABLE "user"
  ADD CONSTRAINT "user_role_id_fkey"
  FOREIGN KEY ("role_id") REFERENCES "role"("id")
  ON DELETE CASCADE ON UPDATE CASCADE;
```

Migration `add_unique_constraint` (matérialisation des contraintes d'unicité métier) :

```
CREATE UNIQUE INDEX "evaluation_evaluator_id_evaluated_id_key"
  ON "evaluation"("evaluator_id", "evaluated_id");

CREATE UNIQUE INDEX "follow_followed_id_follower_id_key"
  ON "follow"("followed_id", "follower_id");
```

16.2. Annexe B — Handler Socket.IO `message:send` (code complet)

Reproduction **verbatim** de `backend/src/realtime/socket.ts:167-347` (dépôt d'équipe, branche `main`) — handler complet de l'envoi d'un message : validation, vérification de participation, persistance Prisma en parallèle, et diffusion multi-rooms. Seule adaptation de mise en page : le bloc est désindenté de quatre espaces, le handler étant imbriqué dans `io.on('connection')` dans le fichier source. Commentaires d'origine conservés en anglais, tels qu'ils figurent dans le livrable.

```
socket.on('message:send', async (payload) => {
  const conversationId = Number(payload?.conversationId);
  const content = String(payload?.message ?? '').trim();

  // Validate input
  if (!Number.isInteger(conversationId) || conversationId <= 0) {
    socket.emit('error', {
      code: 'VALIDATION',
      message: 'Invalid conversationId',
    });
    return;
  }

  if (!content || content.length > 2000) {
    socket.emit('error', {
      code: 'VALIDATION',
      message: 'Invalid message',
    });
    return;
  }

  // Get conversation with participants and message count
  const conv = await prisma.conversation.findUnique({
    where: { id: conversationId },
    select: {
      id: true,
      title: true,
      status: true,
      users: {
        select: {
```

```

      userId: true,
      user: {
        select: {
          id: true,
          firstname: true,
          lastname: true,
          avatarUrl: true,
        },
      },
    },
    _count: {
      select: { messages: true },
    },
  },
});

if (!conv || conv.status === 'Close') {
  socket.emit('error', {
    code: 'FORBIDDEN',
    message: 'Conversation closed',
  });
  return;
}

// Verify that the sender is a participant in the conversation
const userIds = conv.users.map((u) => u.userId);
if (!userIds.includes(userId)) {
  socket.emit('error', {
    code: 'FORBIDDEN',
    message: 'Not a participant',
  });
  return;
}

const participantIds = new Set(userIds);
const receiverId = userIds.find((id) => id !== userId);

if (!receiverId) {
  socket.emit('error', { code: 'FORBIDDEN', message: 'No receiver' });
  return;
}

// Check if this is the first message
const isFirstMessage = conv._count.messages === 0;

// Create the message AND update conversation's updatedAt
const [msg] = await Promise.all([
  prisma.message.create({
    data: {
      conversationId,
      senderId: userId,
      receiverId,
      content,
    },
  },

```

```

    select: {
      id: true,
      content: true,
      createdAt: true,
      sender: {
        select: {
          id: true,
          firstname: true,
          lastname: true,
          avatarUrl: true,
        },
      },
    },
  },
},
}),
prisma.conversation.update({
  where: { id: conversationId },
  data: { updatedAt: new Date() },
}),
]);

// Prepare DTO
const dto: MessageDTO = {
  id: msg.id,
  sender:
    msg.sender && participantIds.has(msg.sender.id)
      ? {
          id: msg.sender.id,
          firstname: msg.sender.firstname,
          lastname: msg.sender.lastname,
          avatarUrl: msg.sender.avatarUrl ?? undefined,
        }
      : undefined,
  content: msg.content,
  timestamp: msg.createdAt.toISOString(),
};

// Emit message to conversation room (for users actively viewing)
io.to(room(conversationId)).emit('message:new', {
  conversationId,
  message: dto,
});

// If first message, notify the receiver about the new conversation
if (isFirstMessage) {
  // Get follow/rating status for the receiver
  const [followStatus, ratingStatus] = await Promise.all([
    prisma.follow.findUnique({
      where: {
        followedId_followerId: {
          followerId: receiverId,
          followedId: userId,
        },
      },
    }),
    prisma.rating.findUnique({

```

```

    where: {
      evaluatorId_evaluatedId: {
        evaluatorId: receiverId,
        evaluatedId: userId,
      },
    },
  },
},
});
});

// Get sender info for the receiver's perspective
const senderUser = conv.users.find((u) => u.userId === userId)?.user;

if (senderUser) {
  // Emit new conversation to receiver only
  io.to(`user:${receiverId}`).emit('conversation:new', {
    conversation: {
      id: conv.id,
      title: conv.title,
      status: conv.status as 'Open' | 'Close',
      participant: {
        id: senderUser.id,
        firstname: senderUser.firstname,
        lastname: senderUser.lastname,
        avatarUrl: senderUser.avatarUrl ?? undefined,
        isFollowing: !!followStatus,
        isRated: !!ratingStatus,
      },
      lastMessage: dto,
    },
  });
}

// Emit conversation update to all participants (via user rooms)
userIds.forEach((participantId) => {
  io.to(`user:${participantId}`).emit('conversation:updated', {
    conversationId,
    lastMessage: dto,
  });
});
});
});
});

```

16.3. Annexe C — Orchestrateur frontend `useMessaging` (139 LOC, complet)

Reproduction **verbatim** et intégrale de `frontend/src/hooks/useMessaging.ts` (dépôt d'équipe, branche `main`) — la façade qui compose les sept hooks spécialisés et écoute les trois events Socket globaux (`conversation:updated`, `conversation:closed`, `conversation:new`).

```

'use client';
import {

```



```

    useConversationList,
    useSelectedConversation,
    useConversationMessages,
    useConversationActions,
    useFollowedUsers,
    useGlobalSocket,
  } from './messaging';
import { useMemo, useEffect } from 'react';
import { ConversationWithMessages } from '@lib/api-types';
import { useAuth } from '@components/providers/AuthProvider';
import { toast } from 'sonner';

export function useMessaging() {
  const { user } = useAuth();

  const {
    conversations,
    isLoading: isConversationLoading,
    addConversation,
    updateConversation,
    updateConversationLastMessage,
    updateConversationStatus,
    removeConversation,
  } = useConversationList();

  const { selectedConvId, setSelectedConvId, selectedConv, clearSelection } =
    useSelectedConversation(conversations);

  const {
    messages,
    isLoading: isLoadingMessages,
    hasMore: hasMoreMessages,
    loadMore: loadMoreMessages,
    addMessage,
    addOptimisticMessage,
  } = useConversationMessages({
    conversationId: selectedConvId,
    limit: 30,
  });

  const { followedUsers, fetchFollowedUsers } = useFollowedUsers();

  const { onConversationUpdate, onConversationClosed, onConversationNew } =
    useGlobalSocket();

  // Listen for lastMessage updates
  useEffect(() => {
    onConversationUpdate((conversationId, lastMessage) => {
      updateConversationLastMessage(conversationId, lastMessage);
    });
  }, [onConversationUpdate, updateConversationLastMessage]);

  // Listen for conversation closures
  useEffect(() => {
    onConversationClosed((conversationId, closedBy) => {

```

```

    updateConversationStatus(conversationId, 'Close');

    if (closedBy && closedBy.id !== user?.id) {
      toast.info(`${closedBy.firstname} à clôturer un échange`);
    }

    if (conversationId === selectedConvId) {
      clearSelection();
    }
  });
}, [
  onConversationClosed,
  updateConversationStatus,
  selectedConvId,
  clearSelection,
  user,
]);

useEffect(() => {
  onConversationNew((conversation) => {
    // Add the new conversation to the list
    addConversation(conversation);

    // Show notification
    toast.info(
      `${conversation.participant?.firstname} a démarré un nouvel échange`,
    );
  });
}, [onConversationNew, addConversation]);

const selectedConvWithMessages: ConversationWithMessages | undefined =
  useMemo(() => {
    if (!selectedConv) return undefined;

    return {
      ...selectedConv,
      messages,
      hasMoreMessages,
      isLoadingMessages,
    };
  }, [selectedConv, messages, hasMoreMessages, isLoadingMessages]);

const {
  handleBack,
  handleViewProfile,
  handleNewConversation,
  handleAddConversation,
  handleRatingUser,
  handleEncloseConversation,
  handleDeleteConversation,
  handleSendMessage,
} = useConversationActions({
  selectedConvId,
  selectedConv: selectedConvWithMessages,
  setSelectedConvId,
});

```

```

    addConversation,
    updateConversation,
    removeConversation,
    clearSelection,
    fetchFollowedUsers,
    addMessage,
    addOptimisticMessage,
  });

  return {
    selectedConvId,
    setSelectedConvId,
    selectedConv: selectedConvWithMessages,
    conversations,
    followedUsers,
    isConversationLoading,
    loadMoreMessages,
    handleBack,
    handleNewConversation,
    handleAddConversation,
    handleSendMessage,
    handleViewProfile,
    handleRatingUser,
    handleEncloseConversation,
    handleDeleteConversation,
  };
}

```

16.4. Annexe D — Composants UI clés de la messagerie

16.4.1. `MessageList.tsx` — affichage virtualisé du fil de discussion

Reproduction **verbatim** et intégrale de

`frontend/src/components/organisms/ConversationPage/MessageThread/MessageList.tsx`

(dépôt d'équipe, branche `main`) — le composant qui rend la liste de messages, gère le spinner de chargement de la pagination cursor-based, l'empty state et la détection de doublons.

```

'use client';
import { RefObject, useEffect } from 'react';
import { MessageBubble } from '@components/molecules/MessageBubble';
import { EmptyState } from '@components/molecules/EmptyState';
import { Loader2 } from 'lucide-react';
import type { Message } from '@lib/api-types';

interface MessageListProps {
  messages: Message[] | undefined;
  currentUserId?: number;
  scrollRef: RefObject<HTMLDivElement | null>;
  isLoading?: boolean;
  hasMore?: boolean;
}

```

```

export function MessageList({
  messages,
  currentUserId,
  scrollRef,
  isLoading = false,
  hasMore = false,
}: MessageListProps) {
  useEffect(() => {
    if (messages) {
      const ids = messages.map((m) => m.id);
      const duplicates = ids.filter((id, index) => ids.indexOf(id) !== index);

      if (duplicates.length > 0) {
        console.error('Duplicate message IDs detected:', duplicates);
        console.error('All messages:', messages);
      }
    }
  }, [messages]);

  return (
    <div
      ref={scrollRef}
      className="flex-1 overflow-y-auto p-6"
      style={{ overflowAnchor: 'none' }} // Important pour éviter le scroll jump
    >
      <div className="flex flex-col">
        {/* Indicateur de chargement en haut */}
        {isLoading && (
          <div className="flex justify-center py-4">
            <Loader2 className="h-6 w-6 animate-spin text-muted-foreground" />
          </div>
        )}

        {/* Indicateur de début de conversation */}
        {!hasMore && messages && messages.length > 0 && (
          <div className="flex justify-center py-4">
            <p className="text-sm text-muted-foreground">
              Début de la conversation
            </p>
          </div>
        )}

        {/* Messages */}
        {messages && messages.length > 0 ? (
          <div className="flex flex-col gap-4">
            {messages.map((m) => (
              <MessageBubble
                key={m.id}
                content={m.content}
                timestamp={m.timestamp}
                isOwn={m.sender?.id === currentUserId}
                sender={m.sender}
              />
            ))}
          </div>
        ) : null}
      </div>
    >
  )
}

```

```

        </div>
      ) : (
        !isLoading && (
          <EmptyState
            title="Aucun message"
            description="Commencez la conversation"
          />
        )
      )}
    </div>
  </div>
);
}

```

16.4.2. MessageInput.tsx — saisie utilisateur et envoi

Reproduction **verbatim** et intégrale de

frontend/src/components/organisms/ConversationPage/MessageThread/MessageInput.tsx

(dépôt d'équipe, branche `main`) — l'input désactivé automatiquement quand la conversation est clôturée (cohérent avec le refus serveur dans le handler `message:send`).

```

'use client';

import { Button } from '@components/atoms/Button';
import { Input } from '@components/atoms/Input';
import { Send } from 'lucide-react';
import { FilterStatus } from '../useConversationState';

interface MessageInputProps {
  value: string;
  onChange: (value: string) => void;
  onSend: () => void;
  isLoading: boolean;
  conversationStatus: FilterStatus;
}

/**
 * Zone de saisie et envoi de message
 */
export function MessageInput({
  value,
  onChange,
  onSend,
  isLoading,
  conversationStatus,
}: MessageInputProps) {
  const handleKeyDown = (e: React.KeyboardEvent) => {
    if (e.key === 'Enter') {
      onSend();
    }
  };
};

```

```

return (
  <div className="border-t bg-background p-4">
    <div className="flex gap-3">
      <Input
        placeholder="Écrivez un message..."
        value={value}
        onChange={(e) => onChange(e.target.value)}
        onKeyDown={handleKeyDown}
        aria-label="Saisir un message"
        disabled={conversationStatus === 'Close'}
      />
      <Button
        onClick={onSend}
        disabled={!value.trim() || conversationStatus === 'Close'}
        className="flex items-center gap-2"
        isLoading={isLoading}
      >
        <Send className="h-4 w-4" />
        <span>Envoyer</span>
      </Button>
    </div>
  </div>
);
}

```

16.5. Annexe E — Plan de tests détaillé de la messagerie

Le plan de tests est détaillé en section 10 ; les jeux d'essai de la fonctionnalité représentative figurent en section 11.

16.6. Annexe F — Maquettes Figma de la messagerie

Maquette : Liste conversations

Maquette : Thread ouvert

Maquette : Création conversation

Maquette : Évaluation post-clôture

16.7. Liens utiles

- **Application en production** : skill-swap.fr

La documentation Arc42 et le guide utilisateur Diátaxis publiés ci-dessous constituent une **extension post-projet, hors périmètre du livrable d'équipe** :

- **Documentation technique Arc42** : skillswap-docs.vercel.app
- **Guide utilisateur (Diátaxis)** : skillswap-guide.vercel.app
- **Repo GitHub doc** : [Sojeremy/documentation-skillswap](https://github.com/Sojeremy/documentation-skillswap)